

Strukture podataka i algoritmi

Algoritmi sortiranja 1

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Sortiranje zamenom (Bubble sort)
2. Sortiranje umetanjem (Insertion sort)
3. Sortiranje selekcijom (Selection Sort)
4. Sortiranje hipa (Heap Sort)

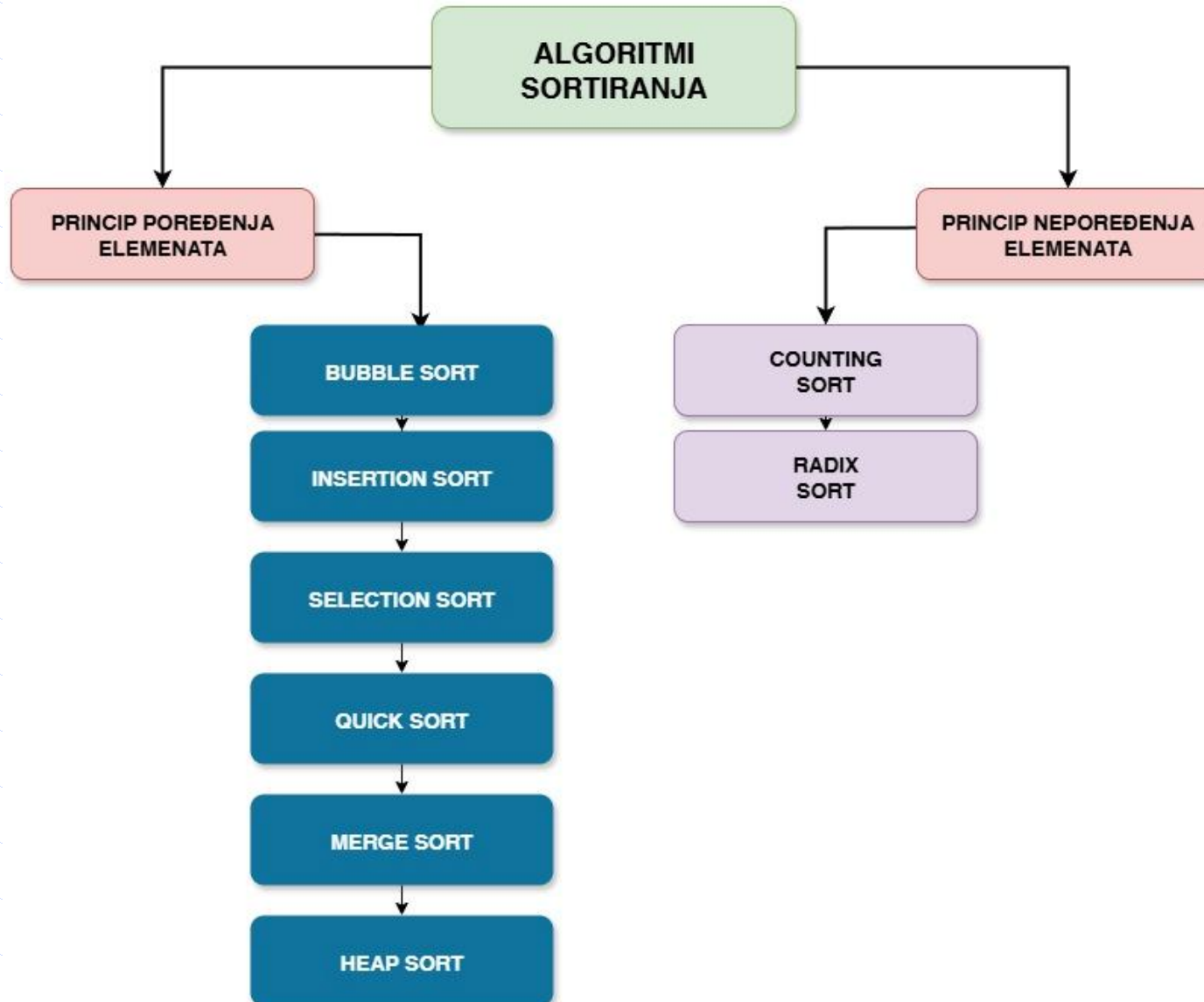
Algoritmi sortiranja

- **Algoritmi sortiranja** imaju značajnu ulogu u računarstvu
- Mogu se klasifikovati na različite načine, ali se najčešće vrši podela na algoritme unutrašnjeg i spoljašnjeg sortiranja
- Unutrašnje sortiranje podrazumeva da se polazna sekvenca u potpunosti nalazi u operativnoj memoriji, dok se kod spoljašnjeg sortiranja, zbog izuzetno velike količine podataka, delovi sekvence nalaze na spoljnoj memoriji (hard-disk, SSD disk ili drugi eksterni uređaj)
- Imajući u vidu da procesor isključivo radi sa podacima u operativnoj memoriji (Fon Nojmanov model računara), algoritmi spoljašnjeg sortiranja zahtevaju kompleksnije postupke od unutrašnjih
- U Python jeziku se za sortiranje, koristi ugrađena funkcija `sorted()` koja vraća novu uređenu sekvencu, odnosno metoda `sort()` koja preuređuje postojeću listu elemenata

Algoritmi sortiranja

- Sortiranje predstavlja preuređivanje datog niza ili liste elemenata prema operatoru poređenja nad tim elementima
- Operator poređenja se koristi za određivanje novog redosleda elemenata u odgovarajućoj strukturi podataka
- Postoje različiti algoritmi sortiranja koji se koriste u strukturama podataka, ali je njihova osnovna klasifikacija zasnovana prema načinu njihovog izvršavanja:
 - a) Algoritmi zasnovani na **upoređivanju elemenata** niza ili liste
 - b) Algoritmi koji **nisu zasnovani na upoređivanju** elemenata niza ili liste
- Algoritmi sortiranja su neophodni u računarstvu jer pojednostavljaju složene probleme i poboljšavaju njihovu efikasnost
- Široko se koriste u zadacima pretraživanja, bazama podataka, strategijama „podeli pa vladaj“ i različitim strukturama podataka

Algoritmi sortiranja - podela



Algoritmi sortiranja

- **In-place sortiranje:** Ovaj tip algoritama sortiranja koristi konstantan prostor za generisanje izlaza (modifikuje se samo dati niz), primeri ovog sortiranja su: Selection sort, Bubble sort, Insertion sort, Heap sort itd...
- **Interno sortiranje:** Svi podaci se smeštaju u glavnu ili internu memoriju, ulazni podaci moraju odgovarati iznosu dodeljene RAM memorije računara
- **Eksterno sortiranje:** kod ovog sortiranja, zbog izuzetno velike količine podataka, delovi sekvenci se nalaze na spoljnoj memoriji (hard-disk, SSD disk...) npr. kod Merge sort algoritma ceo niz ne mora postojati sve vreme u RAM memoriji
- **Stabilno sortiranje:** nastaje kada se dve iste stavke pojavljuju istim redosledom u sortiranim podacima kao u originalnom nizu, primeri su Merge sort, Insertion sort i Bubble sort
- **Hibridno sortiranje:** koristi se prednost više različitih algoritama sortiranja istovremeno, primer je IntroSort algoritma koji koristi Insertion sort i Quick sort

Agoritmi sortiranja – poređenje složenosti

Algoritam sortiranja	Vremenska složenost*	Prostorna složenost	Princip sortiranja
Bubble sort	$O(n^2)$	$O(1)$	Zamena elemenata
Insertion sort	$O(n^2)$	$O(1)$	Umetanje elemenata
Selection sort	$O(n^2)$	$O(1)$	Selekcija elemenata
Quick Sort	$O(n^2)$	$O(\log n)$	Particioniranje
Merge sort	$O(n \log n)$	$O(n)$	Spajanje elemenata
Heap sort	$O(n \log n)$	$O(1)$	Selekcija elemenata
Counting sort	$O(n+m)$	$O(1)$	Učestanost elemenata
Radix sort	$O(d \cdot (n + b))$	$O(n+b)$	Vrednosti cifara elemenata

* Vremenska složenost je prikazana za najgori slučaj

Sortiranje zamenom (Bubble Sort)

Sortiranje zamenom (Bubble sort)

- **Bubble Sort** je jedan od najjednostavnijih algoritama za sortiranje
- Radi na način da se zamenom susednih elemenata najveća vrednost u nizu postavlja na kraj niza
- Iako jednostavan, prosečno vreme izvršavanja ovog algoritma je $O(n^2)$ što ga čini neprikladnim pri radu sa velikim ulaznim podacima (nizovima ili listama)
- Ideja funkcionisanja Bubble Sort algoritma je jednostavna: za datu neuređenu listu ili niz, poredimo susedne elemente i nakon svakog poređenja ih postavljamo u ispravan redosled prema njihovim vrednostima tj. menjamo susedne elemente ako nisu u ispravnom redosledu i ovaj proces se ponavlja $(n-1)$ puta za listu od n elemenata
- U svakoj iteraciji, najveći element liste se pomera na kraj liste. Nakon druge iteracije, drugi najveći element će biti postavljen na preposlednju poziciju na listi i isti postupak se ponavlja dok se niz ne sortira

Bubble Sort

- Korak 1: Sortiramo niz koristeći višestruke iteracije. Nakon prve iteracije, najveći element ide na kraj (svoju ispravnu poziciju)

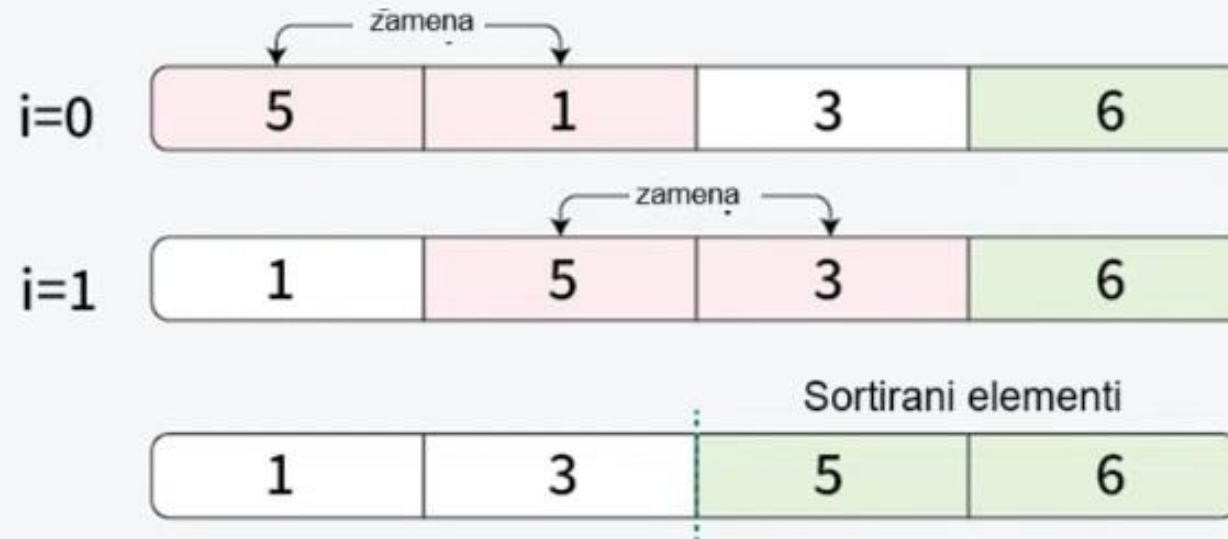


Bubble Sort

- Korak 2: Nakon druge iteracije, drugi najveći element ide na predposlednju poziciju i tako dalje, u svakoj iteraciji obrađujemo samo one elemente koji se već nisu pomerili na ispravnu poziciju, nakon k prolaza, najvećih k elemenata mora biti pomerenih na poslednjih k pozicija

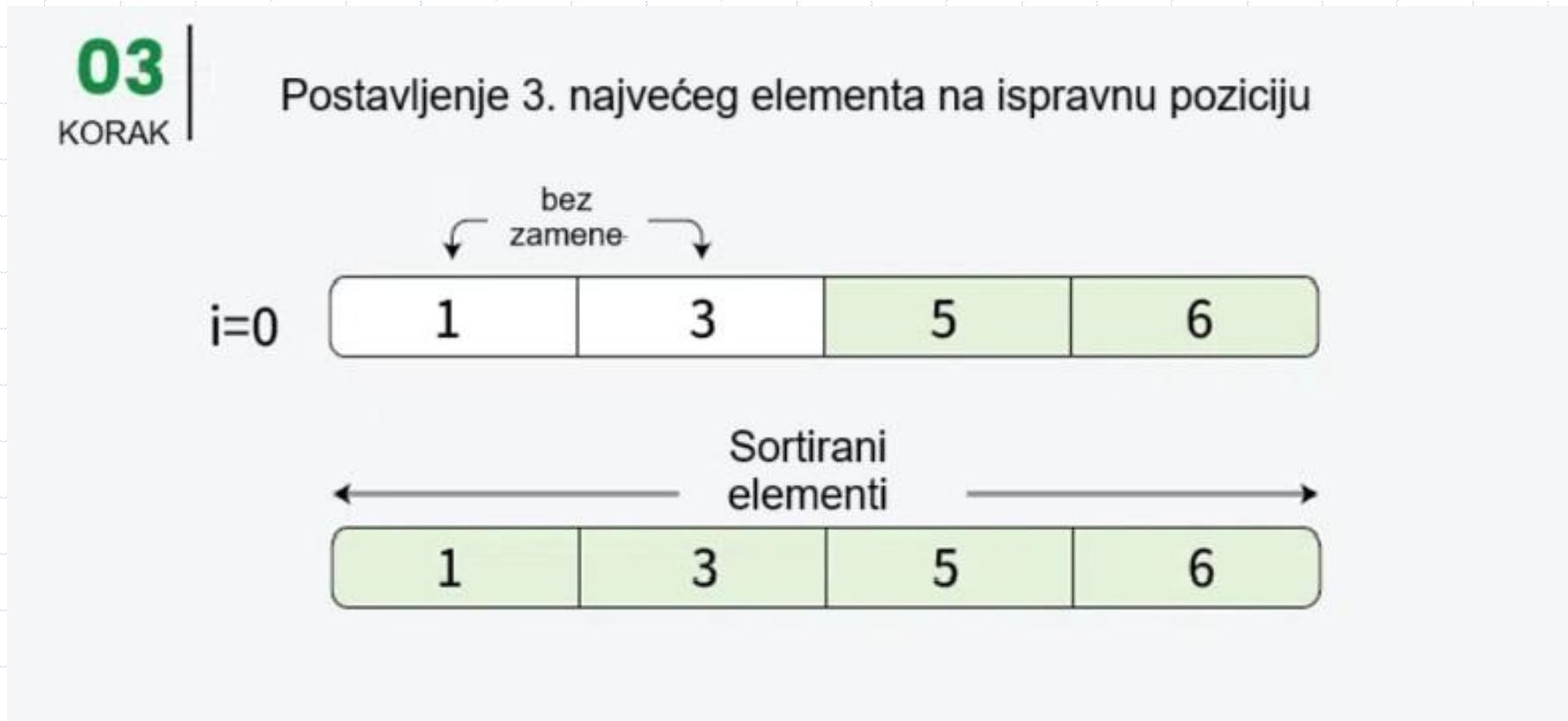
02
KORAK

Postavljanje 2. najvećeg elementa na ispravnu poziciju



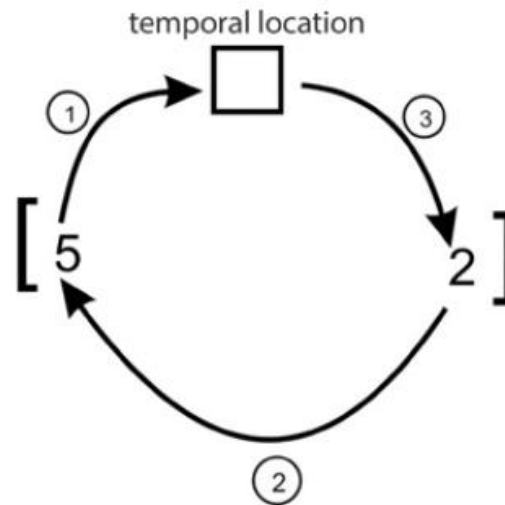
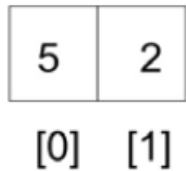
Bubble Sort

- Korak 3: Tokom iteracije, razmatramo preostale elemente i upoređujemo sve susedne elemente i menjamo mesta ako je veći element ispred manjeg elementa, prolazom kroz celu listu dobijamo najveći (među preostalim elementima) na njegovoj ispravnoj poziciji



Bubble Sort

- PRIMER 1:** Sortiranje 2 elementa



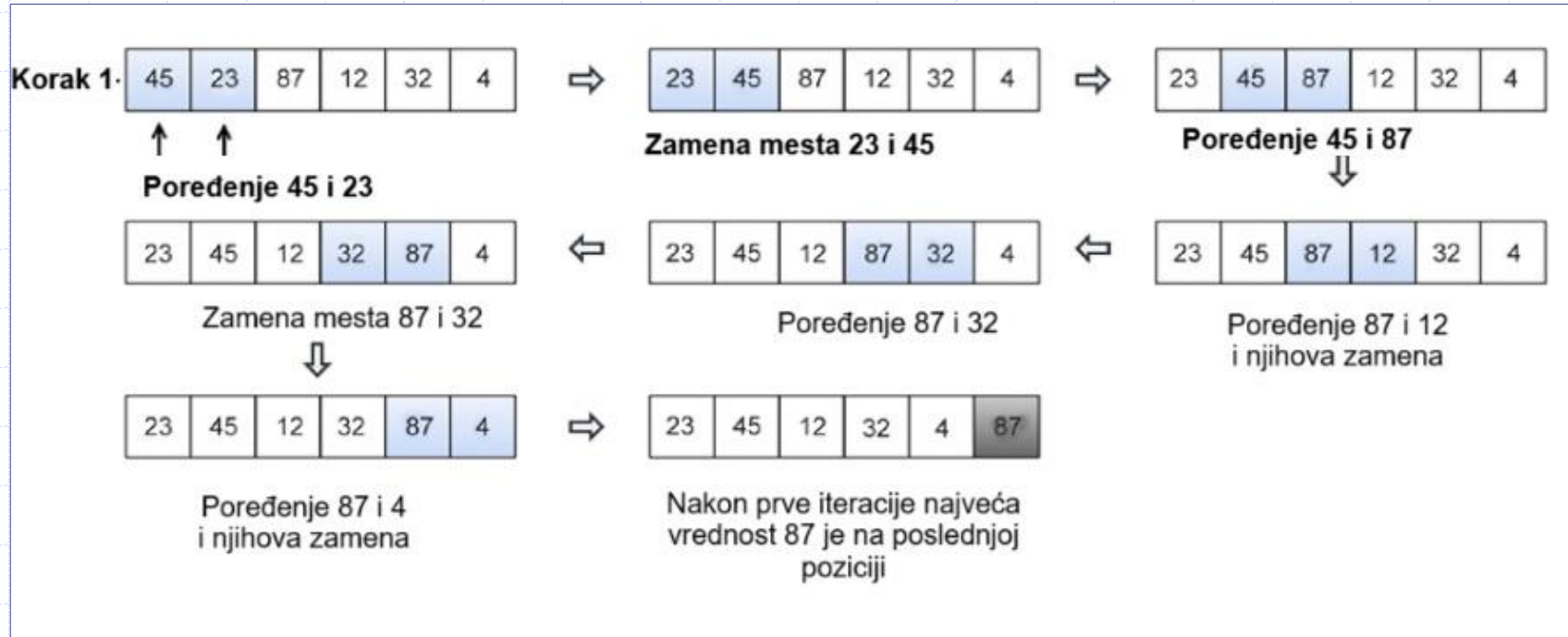
```
unordered_list = [5, 2]
temp = unordered_list[0]
unordered_list[0] = unordered_list[1]
unordered_list[1] = temp
print(unordered_list)
```

[2, 5]

- Za sortiranje dva elementa, prvo upoređujemo brojeve 5 i 2
- Pošto je 5 veće od 2, znači da nisu u ispravnom redosledu, i moraju da zamene pozicije
- Da bismo zamenili pozicije brojeva 5 i 2, prvo pomeramo element sačuvan na indeksu 0 u privremenu promenljivu (korak 1)
- Zatim se element sačuvan na indeksu 1 kopira u indeks 0 (korak 2)
- Na kraju se prvi element sačuvan u privremenoj promenljivoj vraća na indeks 1 (korak 3)
- Prvo se element 5 kopira u privremenu promenljivu `temp` zatim se element 2 pomera u indeks 0 i na kraju 5 se pomera iz `temp` u indeks 1. Lista će sada sadržati elemente u formi [2, 5]

Bubble Sort

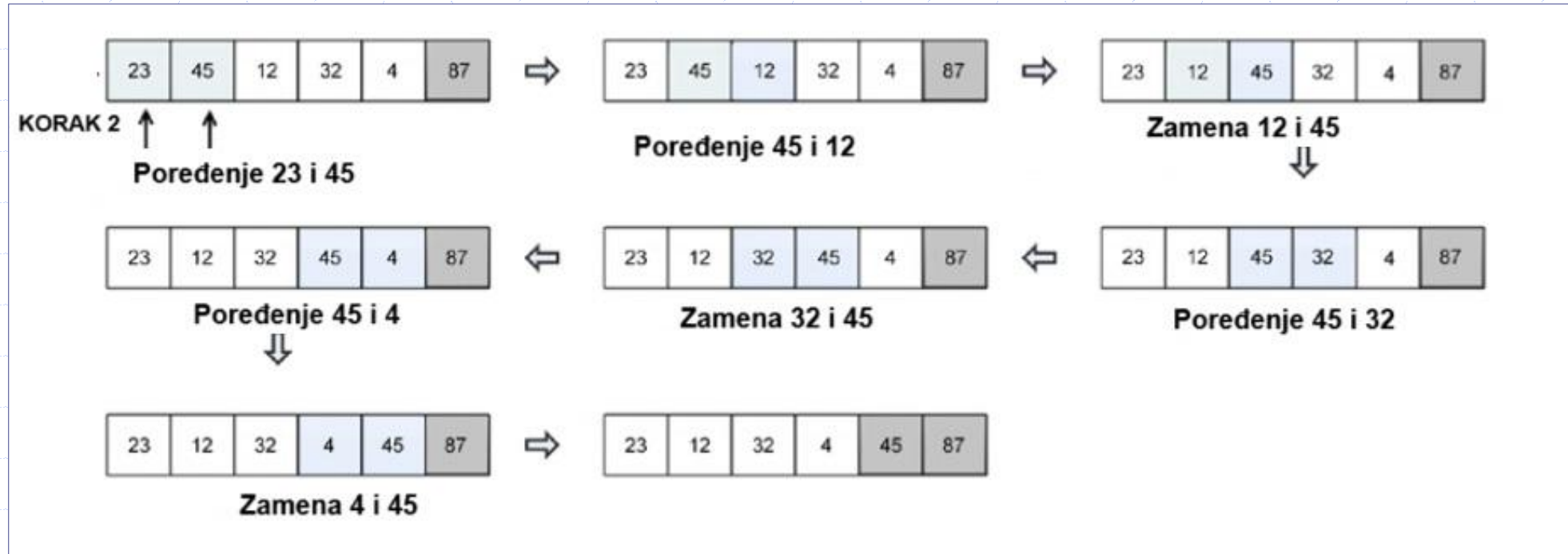
- PRIMER 2: Sortiranje liste od 6 elemenata [45, 23, 87, 12, 32, 4]



- U prvoj iteraciji, porede se prva dva elementa 45 i 23, menjaju mesta jer 45 treba da bude iza 23
- Zatim poredimo sledeće susedne vrednosti 45 i 87
- Pošto je $87 > 45$, ne moramo ih menjati jer su u ispravnom redosledu
- Nakon prve iteracije sortiranja, najveći element 87 se nalazi na poslednjoj poziciji liste

Bubble Sort

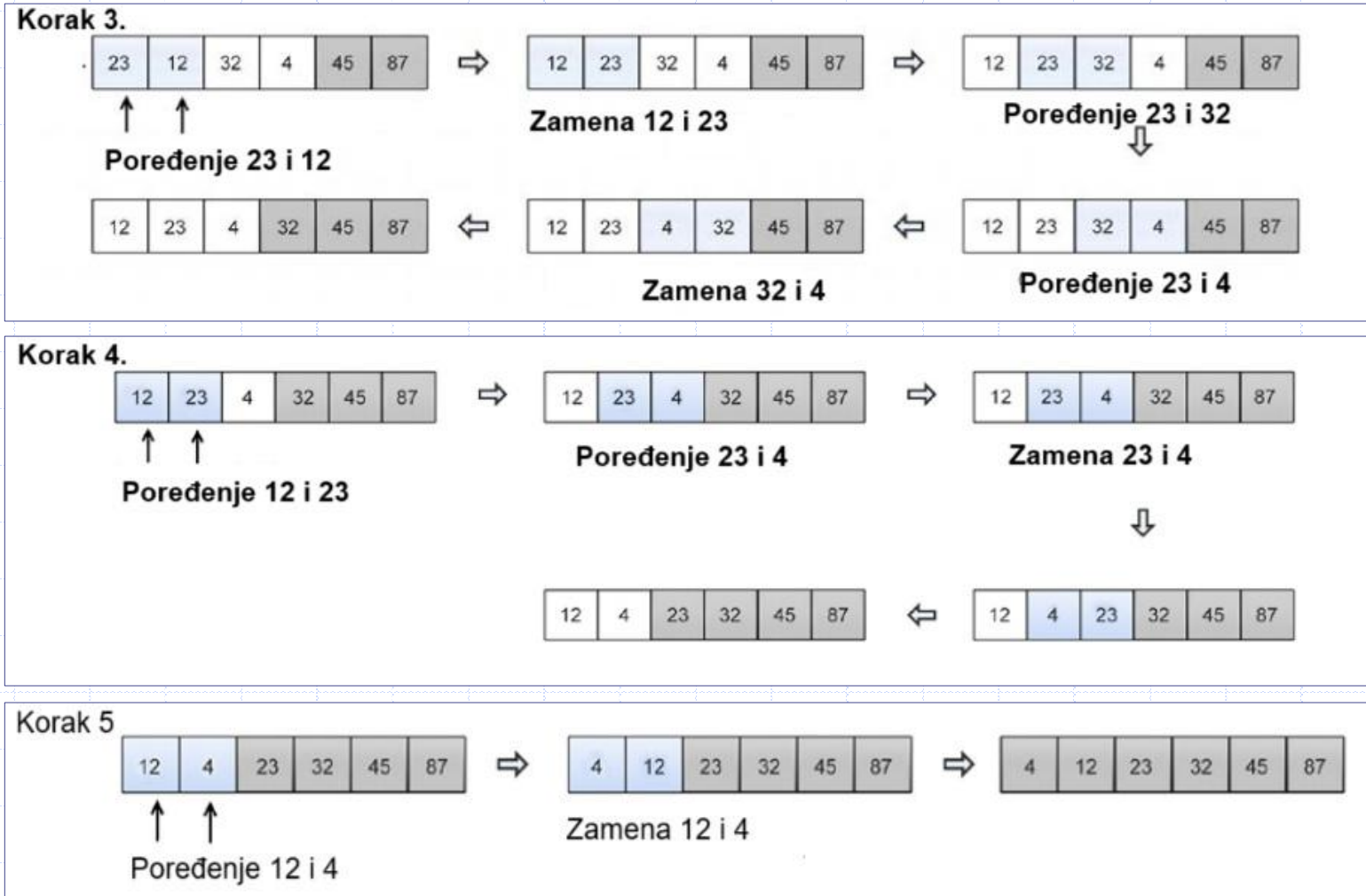
- PRIMER 2: Sortiranje liste od 6 elemenata [45, 23, 87, 12, 32, 4]



- Nakon druge iteracije, potrebno je rasporediti preostalih $(n-1)$ elemenata, ponavlja se isti postupak upoređujući susedne elemente za preostalih 5 elemenata i nakon druge iteracije, drugi najveći element 45, postavlja se na preposlednju poziciju na listi

Bubble Sort

- PRIMER 2: Sortiranje liste od 6 elemenata [45, 23, 87, 12, 32, 4]



Bubble Sort

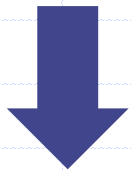
- PRIMER 2: Sortiranje liste od 6 elemenata [45, 23, 87, 12, 32, 4]

```
def bubble_sort(unordered_list):  
    iteration_number = len(unordered_list)-1  
    for i in range(iteration_number,0,-1):  
        for j in range(i):  
            if unordered_list[j] > unordered_list[j+1]:  
                temp = unordered_list[j]  
                unordered_list[j] = unordered_list[j+1]  
                unordered_list[j+1] = temp
```

- Bubble sort se implementira korišćenjem dvostruko ugneždene petlje, jedna petlja je unutar druge
- Unutrašnja petlja poredi i zamenjuje susedne elemente u svakoj iteraciji za datu listu, a spoljašnja petlja prati koliko puta treba ponoviti unutrašnju petlju
- U gornjem kodu prvo se računa koliko puta petlja treba da se pokrene da bi se završile sve zamene, tj. (dužina liste -1), i to je u kodu linija `iteration_number=len(unordered_list)-1`
- Spoljašnja petlja se izvršava se za (dužina liste -1),
- Za svaku iteraciju u unutrašnjoj petlji, upoređujemo susedne elemente koristeći `if` naredbu i proveravamo da li su susedni elementi u ispravnom poretci ili ne
- Za prvu iteraciju, unutrašnja petlja se pokreće `n` puta, za drugu treba da se pokrene `n-1` puta itd...

Bubble Sort

```
1  # Bubble sort algoritam....
2
3  def bubble_sort(array):
4      n = len(array)
5      for i in range(n-1):
6          for j in range(n-i-1):
7              if array[j] > array[j+1]:
8                  array[j], array[j+1] = array[j+1], array[j]
9      return array
10
11 arr = [10, 5, 2, 8, 3]
12 print(f"Input: arr = {arr}")
13 print("Output:", bubble_sort(arr))
14
```



● Input: arr = [10, 5, 2, 8, 3]
Output: [2, 3, 5, 8, 10]

Bubble Sort algoritam sortiranja počinje sa spoljašnjom petljom koja iterira kroz niz da bi se izvršilo više prolaza. U svakom prolazu, unutrašnja petlja upoređuje susedne elemente i zamenjuje ih ako su u pogrešnom redosledu. Do kraja svakog prolaza kroz niz, najveći element će da „istrči“ do svoje ispravne pozicije na kraju niza

- Spoljašnja petlja (i) se kreće od 0 do n-2 (zaključno) jer nakon što se prođe n-1 elemenata, najveći element je na svojoj ispravnoj poziciji
- Unutrašnja petlja (j) se kreće od 0 do n-i-2 (zaključno) jer je najvećih i elemenata već sortirano na kraju niza nakon svakog prolaza
- Ako je trenutni element niza[j] veći od sledećeg elementa niza[j+1], vrši se zamena
- Proces se nastavlja dok se ceo niz ne sortira i više nisu potrebne zamene

Bubble Sort – programski kôd

#Python program za Bubble Sort

```
def bubbleSort(arr):  
    n = len(arr)  
    # Prolazak kroz sve elemente niza  
    for i in range(n):  
        swapped = False  
        # Poslednjih i elemenata je već na svom mestu  
        for j in range(0, n-i-1):  
            # Prolazak kroz niz od 0 do n-i-1  
            # Zameni ako je pronađeni element veći od sledećeg  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
                swapped = True
```

Bubble Sort – programski kôd

```
    if (swapped == False):  
        break
```

Testiranje koda

```
if __name__ == "__main__":  
    arr = [64, 34, 25, 12, 22, 11, 90]  
    bubbleSort(arr)  
    print("Sorted array:")  
    for i in range(len(arr)):  
        print("%d" % arr[i], end=" ")
```

Output

```
Sorted array:  
11 12 22 25 34 64 90
```

Bubble Sort - karakteristike

- Bubble sort nije efikasan algoritam sortiranja, jer je njegova složenost izvršavanja za najgori slučaj $O(n^2)$ i za najbolji slučaj $O(n)$
- Najgori slučaj se javlja kada želimo da sortiramo datu listu u rastućem redosledu, a lista za sortiranje je u opadajućem redosledu, a najbolji slučaj se javlja kada je data lista već sortirana jer tada nema potrebe za zamenom
- Generalno ovaj algoritam ne bi trebalo koristiti za sortiranje velikih listi ili nozova
- Bubble Sort algoritam je pogodan za primene gde performanse nisu bitne ili je dužina liste kratka, i kada se zahteva kratak i jednostavan programski kod
- U računarskoj grafici je popularan zbog svoje sposobnosti da detektuje veoma male greške (kao što je zamena samo dva elementa) u nizovima i da ih popravi sa linearnom složenošću $O(n)$ (koristi se u algoritmu za popunjavanje poligona, gde se granične linije sortiraju po x-koordinati na određenoj liniji skeniranja i sa povećanjem y njihov redosled se menja (tj. dva elementa se zamenjuju) samo na presecima dve linije



Bubble Sort algoritam sortiranja

PRIMER

7

4

6

-2

1



7

4

6

-2

1





4

7

6

-2

1

4

7

6

-2

1





4

6

7

-2

1

4

6

7

-2

1





4

6

-2

7

1

4

6

-2

7

1





4

6

-2

1

7



4

6

-2

1

7



4

6

-2

1

7





4

-2

6

1

7



4

-2

6

1

7





4

-2

1

6

7



4

-2

1

6

7



4

-2

1

6

7





-2

4

1

6

7



-2

4

1

6

7





-2

1

4

6

7



-2

1

4

6

7



-2

1

4

6

7



-2

1

4

6

7



-2

1

4

6

7



-2

1

4

6

7

Sortiranje umetanjem (Insertion Sort)

Sortiranje umetanjem (Insertion sort)

- Insertion sort je algoritam za sortiranje sa prosečnom vremenskom složnošću $O(n^2)$
- Pogodan je samo za sortiranje manjih nizova ili listi ulaznih podataka
- Dizajniran je za efikasno sortiranje elemenata postepenim formiranjem sortiranog podniza unutar nesortiranog niza
- Glavna ideja ovog algoritma je ispitati svaki nesortirani element i umetnuti ga na njegovu ispravnu poziciju unutar sortiranog podniza, čime se sortirani podniz postupno povećava
- Algoritam formira dve podliste (podniza), jednu koja je sortirana i jednu koja nije sortirana, u kojima se elementi dodaju jedan po jedan iz nesortirane podliste u sortiranu
- Uzimaju se elementi iz nesortirane podliste i ubacuju na ispravnu poziciju u sortiranoj podlisti

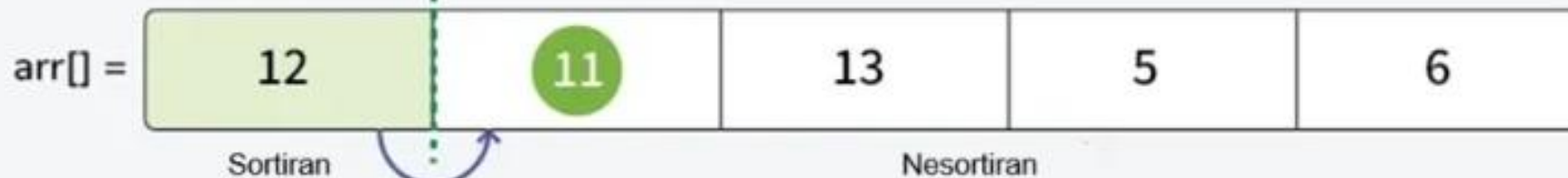
Insertion Sort

- Korak 1: Počinje se sa drugim elementom niza, pretpostavka je da je prvi element već sortirani, poredi se drugi element sa prvim, ako je manji tada ga zameniti sa prvim

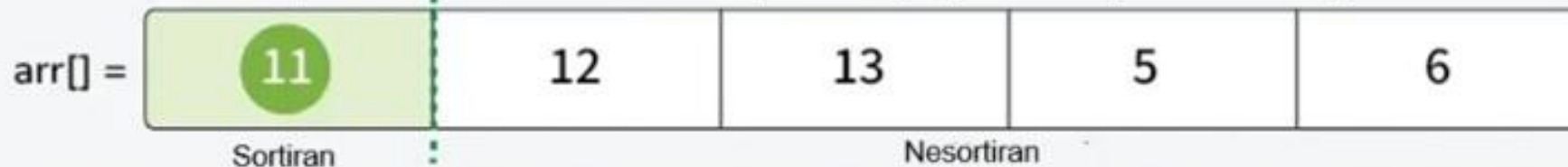
1. prolaz

Key = 11

Poređenje Key sa 12, pošto je 12 veće od Key, pomera se desno za jednu poziciju i menjaju mesta



Nema preostalih elemenata za poređenje pa se Key umeće na početak



Insertion Sort

- Korak 2: Uzima se sledeći element nesortiranog podniza i upoređuje sa elementima sortiranog podniza, pošto je 13 veće od 11 i 12 nema zamene i pomeranja

2. prolaz

Key = 13

Poredi se Key sa 12, poredak je ispravan, pa nema pomeranja

arr[] =



Sortiran

Nesortiran

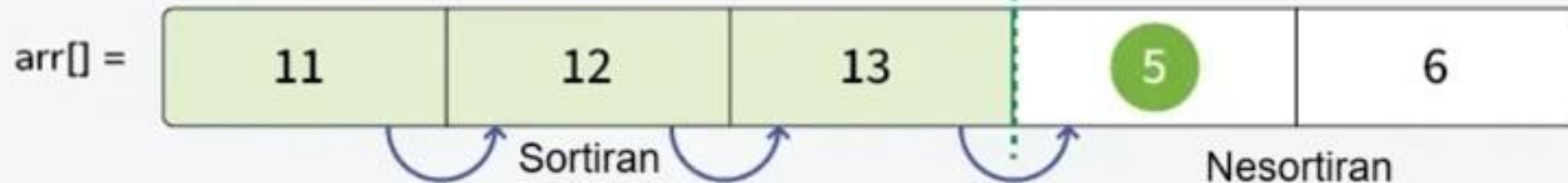
Insertion Sort

- Korak 3:** Prelazi se na sledeći element nesortiranog podniza, 5 se poredi počevši od broja 13 u sortiranom podnizu i pošto je manji od svih, postavlja se na prvu poziciju

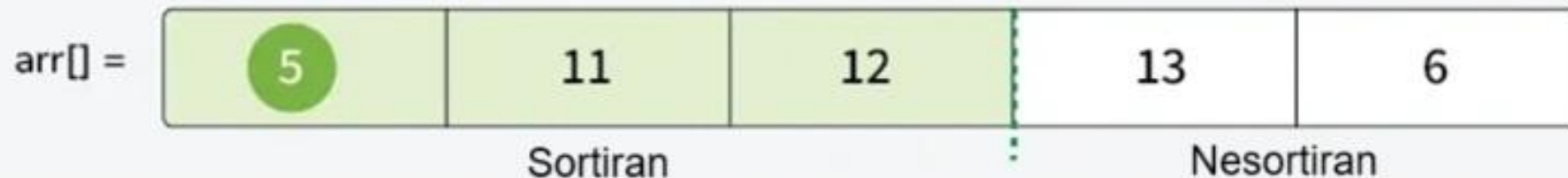
3. prolaz

Key = 5

Poredi se Key sa svim elementima sortiranog podniza počevši od 13, ako je element veći, pomera se udesno



Nema više elemenata za poređenje, Key se umeće na početak



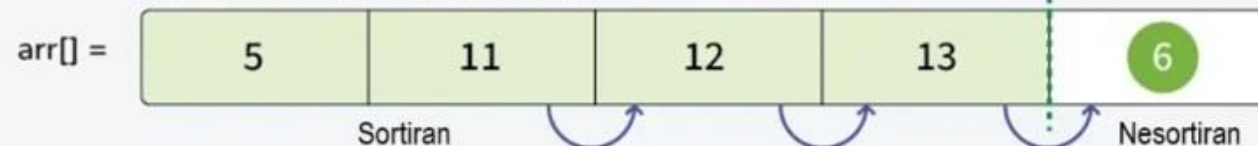
Insertion Sort

- Koraci 4 i 5: Prelazi se na poslednji element nesortiranog podniza, 6 se poredi počevši od broja 13 u sortiranom podnizu i pošto je manji od svih, postavlja se na drugu poziciju iza broja 5

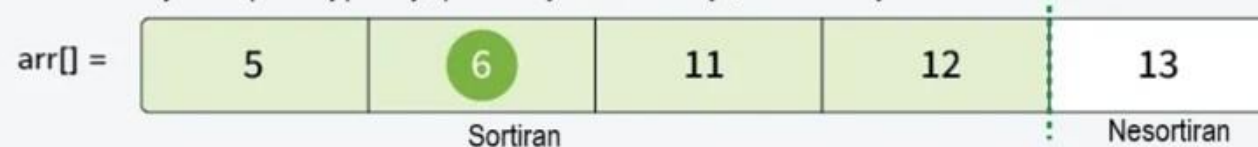
4. prolaz

Key = 6

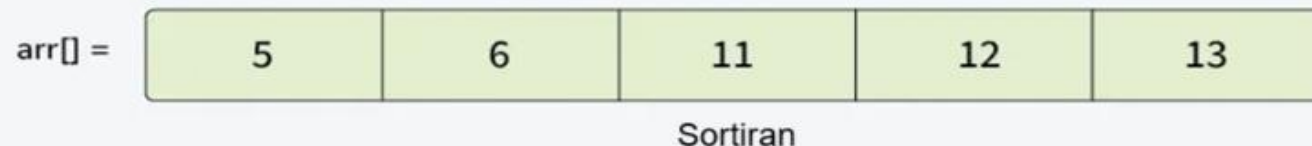
Key se poredi sa svim elementima sortiranog podniza, počevši od 13, ako je element veći, pomera se desno



5 je na ispravnoj poziciji, pomeranje se zaustavlja, 6 se ubacuje nakon 5

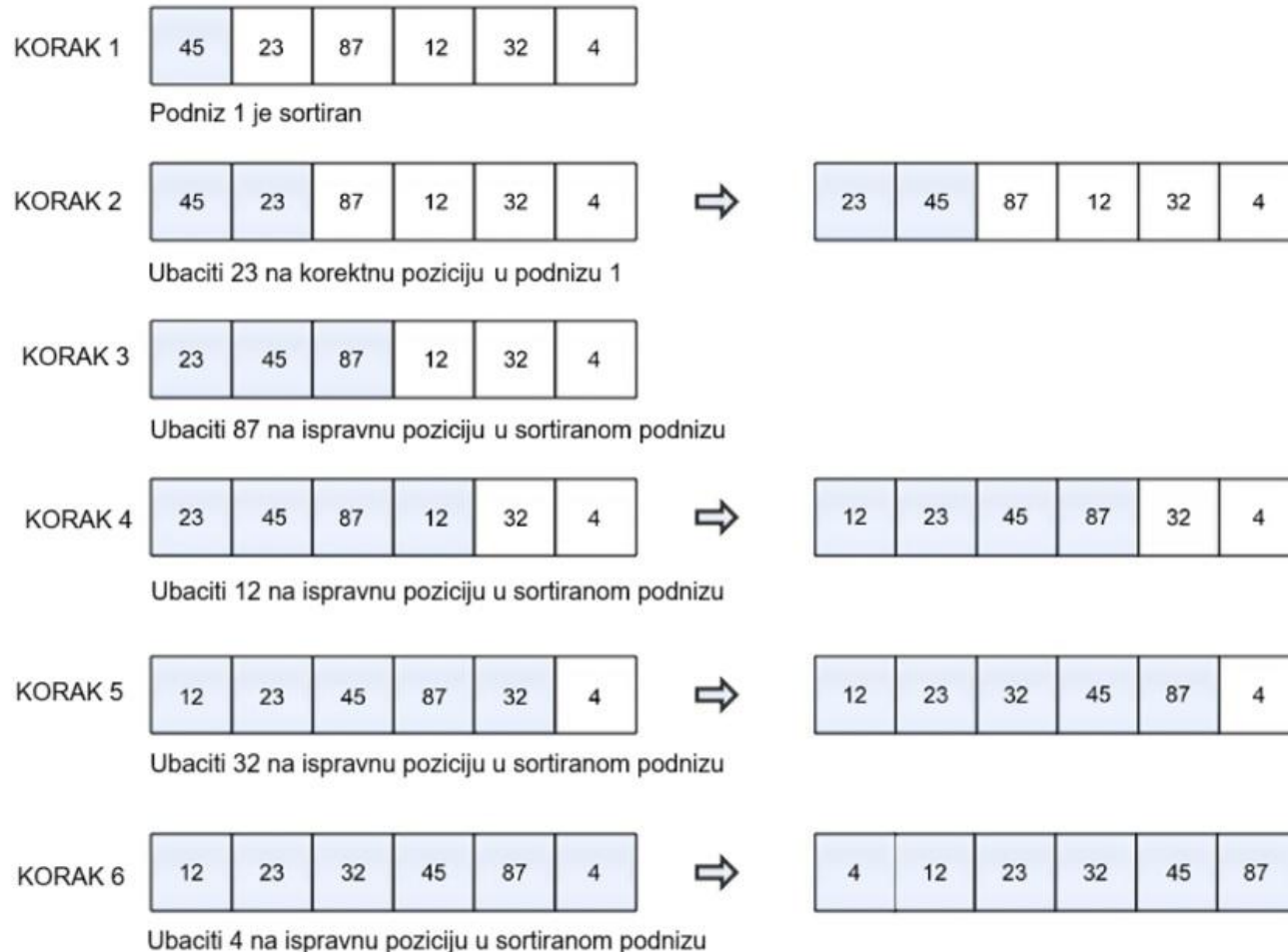


Sortirani podniz sadrži sve elemente početnog niza, niz je sortiran



Insertion Sort

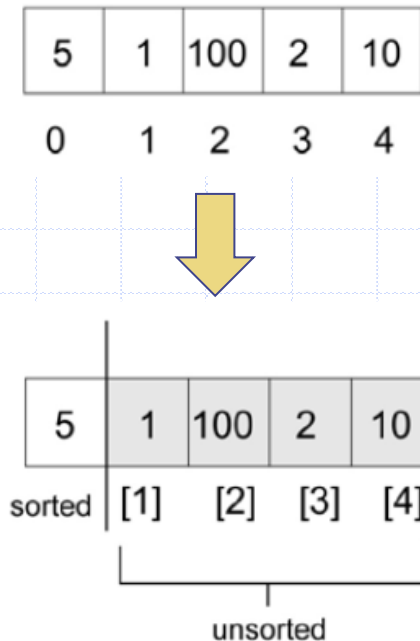
- PRIMER 1: Sortiranje niza od 6 elemenata [45, 23, 87, 12, 32, 4]



- Počinje se od prvog elementa, pod pretpostavkom da je sortiran
- Uzima se sledeći element, 23, iz nesortiranog podniza i ubacuje se na ispravnu poziciju u sortiranom podnizu
- U sledećoj iteraciji uzima se treći element 87 iz nesortirane podliste i ubacuje se u sortirani podniz na ispravnu poziciju
- Ponavljaju se iste iteracije sve dok se svi elementi ne nađu u sortiranoj podlisti

Insertion Sort

- PRIMER 2 : Sortiranje niza od 5 elemenata [5, 1, 100, 2, 10]



```
def insertion_sort(unsorted_list):  
    for index in range(1, len(unsorted_list)):  
        search_index = index  
        insert_value = unsorted_list[index]  
        while search_index > 0 and unsorted_list[search_index-1] > insert_  
value :  
            unsorted_list[search_index] = unsorted_list[search_index-1]  
            search_index -= 1  
        unsorted_list[search_index] = insert_value
```

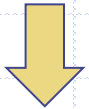
- Algoritam počinje korišćenjem **for** petlje za izvršavanje između indeksa 1 i 4
- Počinje se od indeksa 1 jer je sačuvani element u sortiranom podnizu sa indeksom 0, a elementi između indeksa 1 i 4 su iz nesortiranog podniza

Na početku svakog izvršavanja **for** petlje, element `unsorted_list[index]` se čuva u promenljivoj `insert_value`

Insertion Sort

- PRIMER 2 : Sortiranje niza od 5 elemenata [5, 1, 100, 2, 10]

5	1	100	2	10
0	1	2	3	4



5	1	100	2	10
sorted	[1]	[2]	[3]	[4]

unsorted



1	5	100	2	10
[0]	[1]	[2]	[3]	[4]

sorted portion of list

unsorted portion of list

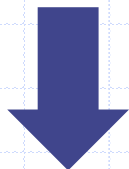
search_index

```
def insertion_sort(unsorted_list):  
    for index in range(1, len(unsorted_list)):  
        search_index = index  
        insert_value = unsorted_list[index]  
  
        while search_index > 0 and unsorted_list[search_index-1] > insert_value :  
            unsorted_list[search_index] = unsorted_list[search_index-1]  
            search_index -= 1  
            unsorted_list[search_index] = insert_value
```

- Kada se pronađe odgovarajuća pozicija u sortiranom podnizu, `insert_value` će biti sačuvan na tom indeksu u sortiranom podnizu
- `search_index` se koristi za `while` petlju, odnosno označava gde tačno pronaći sledeći element koji treba ubaciti u sortiranu podlistu
- Petlja `while` prolazi kroz niz unazad, vođena sa dva uslova. Prvo, ako je `search_index > 0`, znači da postoji više elemenata u sortiranom delu liste, drugo da bi se petlja `while` pokrenula, `unsorted_list[search_index-1]` mora biti veće od promenljive `insert_value`
- Niz `unsorted_list[search_index-1]` će uraditi sledeće:
 - a) pokazivati na element, neposredno pre `unsorted_list[search_index]`, pre nego što se `while` petlja izvrši prvi put
 - b) pokazivati na jedan element pre `unsorted_list[search_index-1]`, nakon što se `while` petlja izvrši prvi put
- `while` petlja će biti izvršena jer je `5 > 1`, dok u `while` petlji element na `unsorted_list[search_index-1]` je sačuvan na `unsorted_list[search_index]`, pa `search_index -= 1` pomera obilazak liste unazad dok se ne dobije vrednost 0

Insertion Sort

```
1  # Insertion Sort algoritam....
2
3  def insertion_sort(array):
4      for i in range(1, len(array)):
5          j = i
6
7          while j > 0 and array[j] < array[j-1]:
8              array[j], array[j-1] = array[j-1], array[j]
9              j -= 1
10
11     return array
12
13 arr = [10, 5, 2, 8, 3]
14 print(f"Input: arr = {arr}")
15 print("Output:", insertion_sort(arr))
```



- Input: arr = [10, 5, 2, 8, 3]
Output: [2, 3, 5, 8, 10]

Insertion Sort je jednostavan algoritam sortiranja zasnovan na poređenju koji formira sortirani niz iterativnim umetanjem svakog elementa na njegovu odgovarajuću poziciju unutar već sortiranog dela niza. Algoritam počinje sa spoljašnjom petljom koja iterira od drugog do poslednjeg elementa niza, razmatra trenutni element (`array[i]`) i pomera ga na ispravnu poziciju unutar sortiranog dela niza

- Unutrašnja `while` petlja počinje od indeksa tekućeg elementa (`j=i`) i nastavlja ka početku niza (`j>0`) sve dok je tekući element manji od susednog (`array[j]<array[j-1]`)
- U svakoj iteraciji unutrašnje petlje, ako je uslov zadovoljen, vrši se zamena da bi se tekući element pomerio za jednu poziciju unazad
- Proces se nastavlja sve dok tekući element ne bude na ispravnoj poziciji, a zatim se spoljašnja petlja pomera na sledeći element

Insertion Sort – programski kôd

```
# Funkcija za sortiranje Insertion sort
def insertionSort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Pomeranje elemenata niza arr[0..i-1], koji su
        # veći od Key, za jednu poziciju iza tekuće
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

# Funkcija za ispis niza veličine n
def printArray(arr):
    for i in range(len(arr)):
        print(arr[i], end=" ")
    print()

# Driver kod
if __name__ == "__main__":
    arr = [12, 11, 13, 5, 6]
    insertionSort(arr)
    printArray(arr)
```

Output



```
5 6 11 12 13
```

Insertion Sort

- U algoritmu Insertion sort se izvršava n iteracija spoljašnje for petlje, a u svakoj od tih iteracija za $i=1,2,3\dots n$ izvršava se najviše $i-1$ iteracija unutar while petlje
- Zbog čega najviše $i-1$ iteracija ugneždene while petlje? Zato što promenljiva j dobija početnu vrednost jednaku vrednosti promenljive i pre početka while petlje, a j se smanjuje za jedan tokom svake iteracije do završetka petlje, pa se while petlja sigurno završava kada je $j=1$; u najgorem slučaju iteracije while petlje se izvršavaju za sve uzastopne vrednosti j od i do 2 , pa je broj tih iteracija najviše $i-1$
- Za najgori slučaj, kada je ulazni niz u opadajućem redosledu, Insertion Sort algoritam se izvršava kao kvadratna zavisnost $O(n^2)$
- Najbolji slučaj je kada je ulazni niz u rastućem redosledu, pa je broj iteracija while petlje jednak 0 , i izvršavanje je linearno $O(n)$
- Algoritam Insertion Sort je jednostavan za implementaciju
- Za velike ulazne podatke je neefikasan, ali za nizove 10-20 elemenata je potpuno prihvatljiv

Insertion Sort

- Adaptivni algoritam što znači da se ubrzava kako se povećava sortirani deo niza
- Stabilan tip sortiranja, čuva se relativni redosled istih sortiranih elemenata
- In-place algoritam – ne zahteva dodatnu memoriju
- Online algoritam – može da vrši sortiranje niza koji se preuzima direktno sa weba
- Varijanta Insertion Sort algoritma je „shell sort“
- Insertion Sort zahteva više programskog koda jer unutrašnja while petlja može zahtevati pomeranje velikih delova sortiranog dela niza
- Selection Sort algoritam može biti bolje rešenje od Insertion Sort algoritma u slučajevima kada je upis u memoriju znatno skuplja opcija od čitanja iz nje, kao npr. u slučaju korišćenja fleš memorija

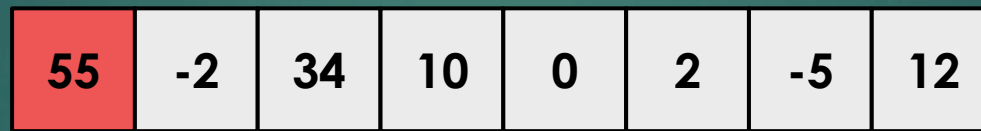


Insertion Sort algoritam sortiranja

PRIMER



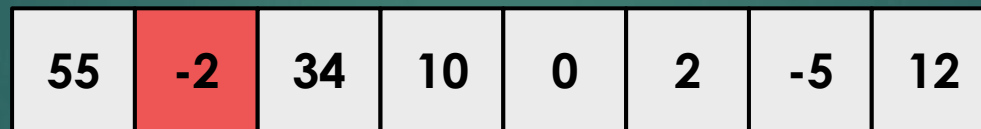
55	-2	34	10	0	2	-5	12
----	----	----	----	---	---	----	----



55	-2	34	10	0	2	-5	12
----	----	----	----	---	---	----	----



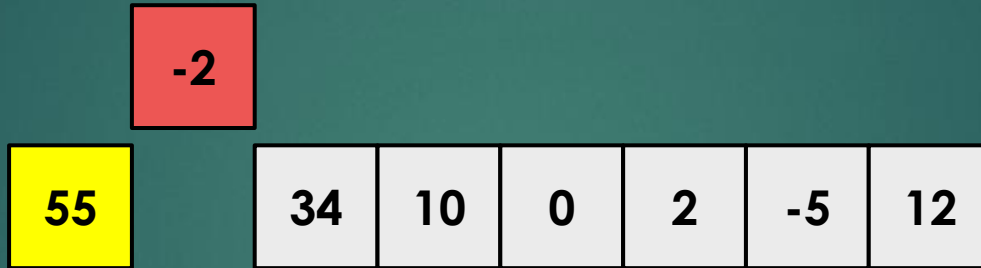
55	-2	34	10	0	2	-5	12
----	----	----	----	---	---	----	----



55	-2	34	10	0	2	-5	12
----	----	----	----	---	---	----	----



	-2						
55		34	10	0	2	-5	12



55	-2	34	10	0	2	-5	12
----	----	----	----	---	---	----	----

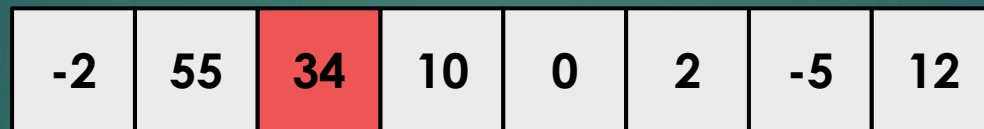
-2

55	34	10	0	2	-5	12
----	----	----	---	---	----	----

-2	55	34	10	0	2	-5	12
----	----	----	----	---	---	----	----

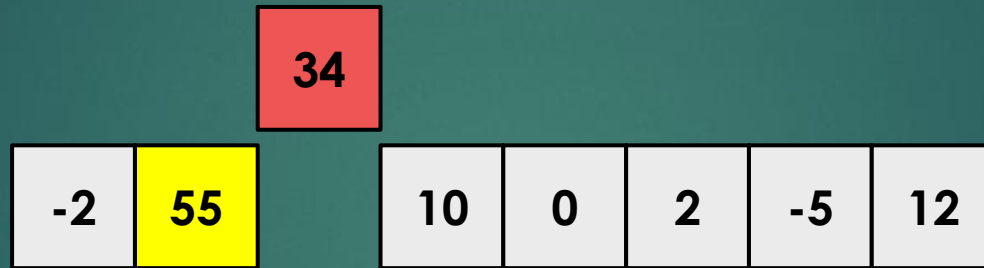


-2	55	34	10	0	2	-5	12
----	----	----	----	---	---	----	----



-2	55	34	10	0	2	-5	12
----	----	----	----	---	---	----	----

		34					
-2	55		10	0	2	-5	12

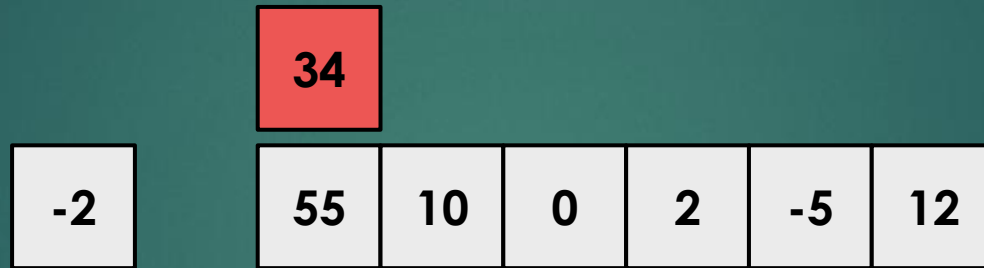


	34					
-2	55	10	0	2	-5	12

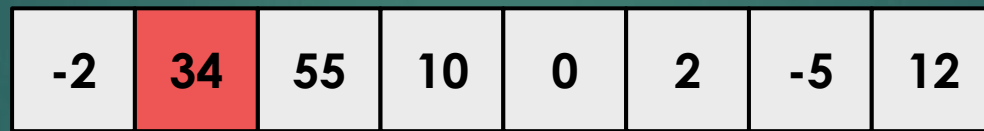
-2

34

55	10	0	2	-5	12
----	----	---	---	----	----



	34						
-2	55	10	0	2	-5	12	



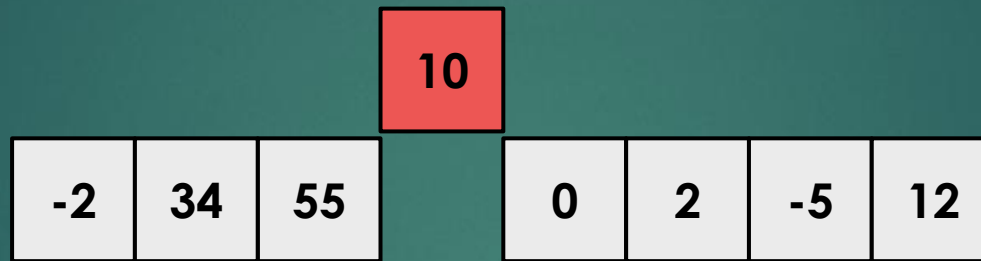
-2	34	55	10	0	2	-5	12
----	----	----	----	---	---	----	----



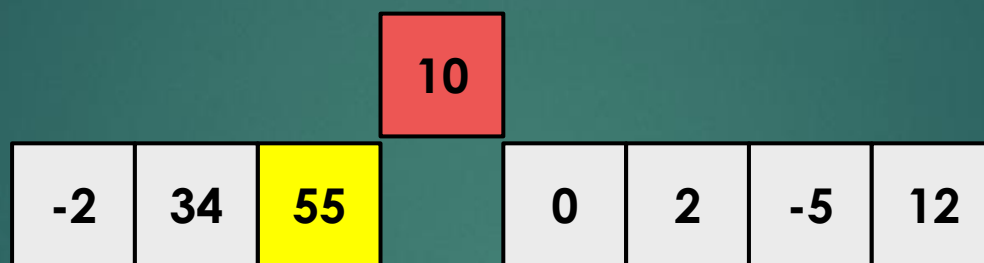
-2	34	55	10	0	2	-5	12
----	----	----	----	---	---	----	----



-2	34	55	10	0	2	-5	12
----	----	----	----	---	---	----	----



-2	34	55		0	2	-5	12
			10				



-2	34	55		10	0	2	-5	12
----	----	----	--	----	---	---	----	----



		10				
-2	34	55	0	2	-5	12



-2	34
----	----

10				
55	0	2	-5	12

		10				
-2	34	55	0	2	-5	12



-2

34	55	0	2	-5	12
-----------	-----------	----------	----------	-----------	-----------

10





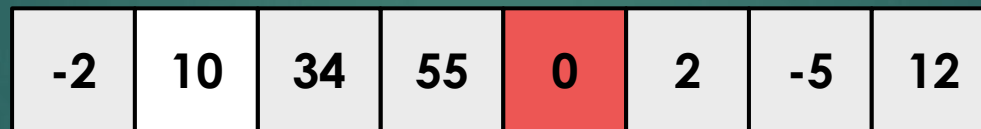
		10				
-2	34	55	0	2	-5	12



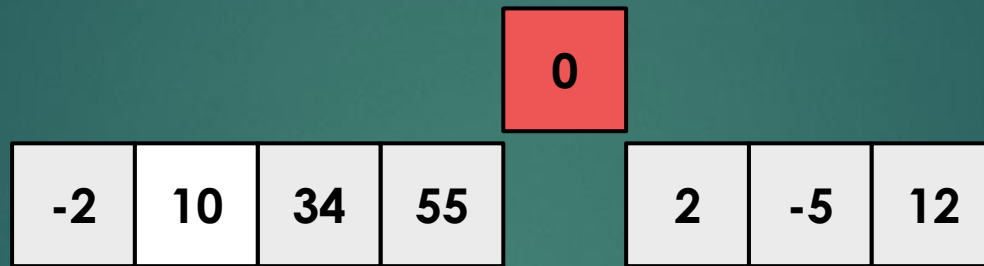
-2	10	34	55	0	2	-5	12
----	----	----	----	---	---	----	----

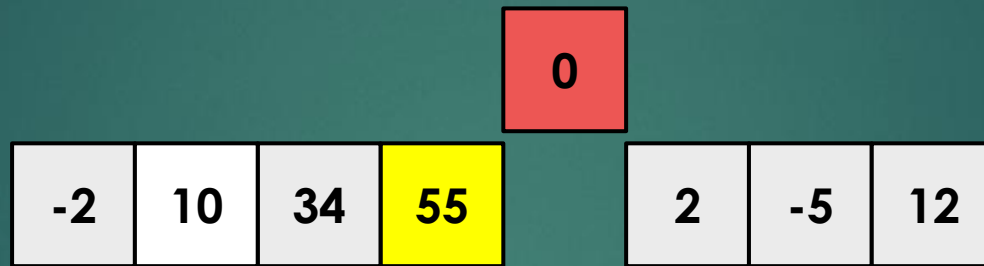


-2	10	34	55	0	2	-5	12
----	----	----	----	---	---	----	----



-2	10	34	55	0	2	-5	12
----	----	----	----	---	---	----	----





-2	10	34
----	----	----

0

55	2	-5	12
----	---	----	----

-2	10	34
----	----	----

0

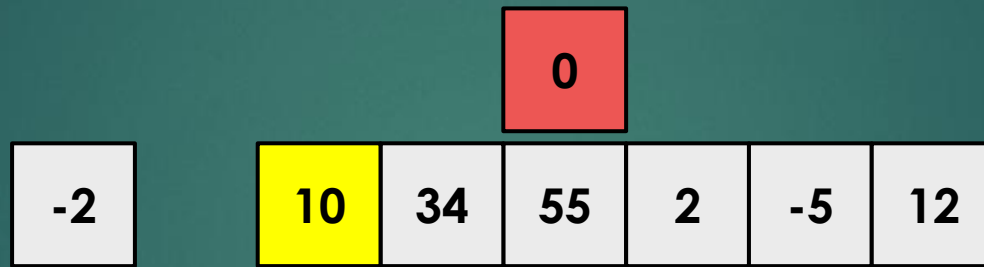
55	2	-5	12
----	---	----	----

-2	10
----	----

					0					
34	55	2	-5	12						

-2	10
----	----

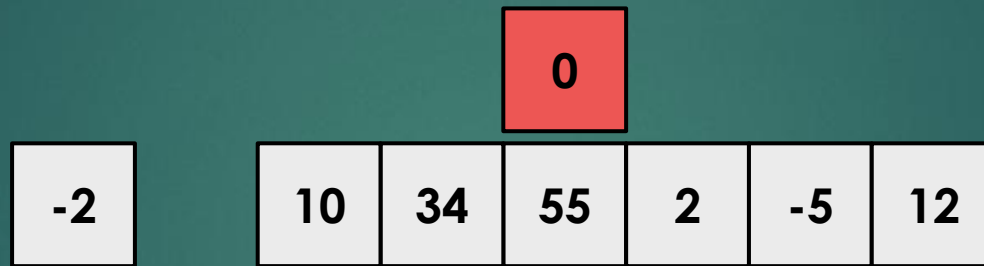
0				
34	55	2	-5	12



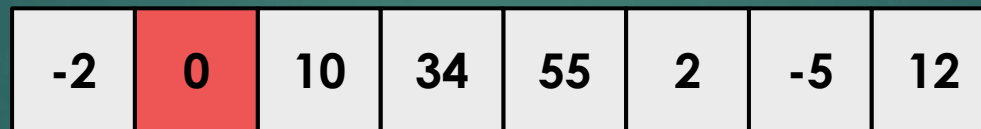
-2

10	34	55	2	-5	12
----	----	----	---	----	----

0



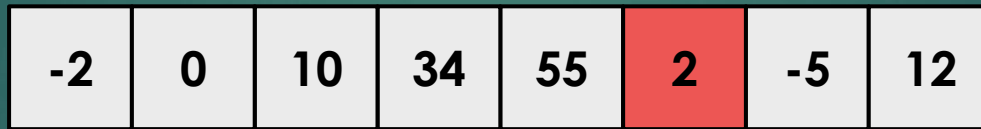
			0			
-2	10	34	55	2	-5	12



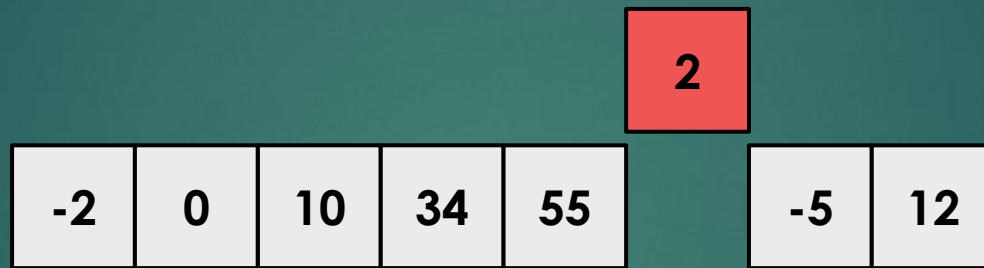
-2	0	10	34	55	2	-5	12
----	---	----	----	----	---	----	----

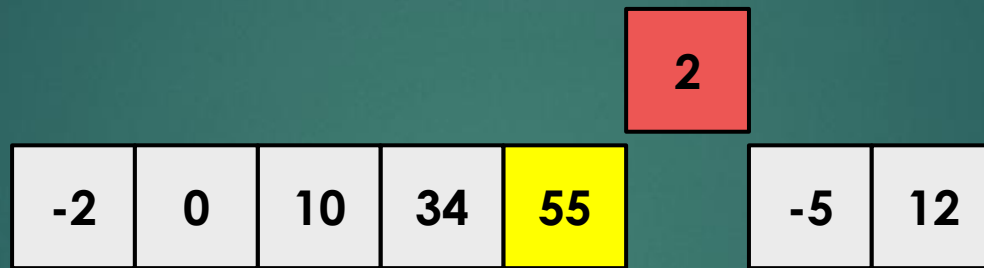


-2	0	10	34	55	2	-5	12
----	---	----	----	----	---	----	----



-2	0	10	34	55	2	-5	12
----	---	----	----	----	---	----	----





-2	0	10	34
----	---	----	----

2		
55	-5	12

-2	0	10	34
----	---	----	----

2

55	-5	12
----	----	----

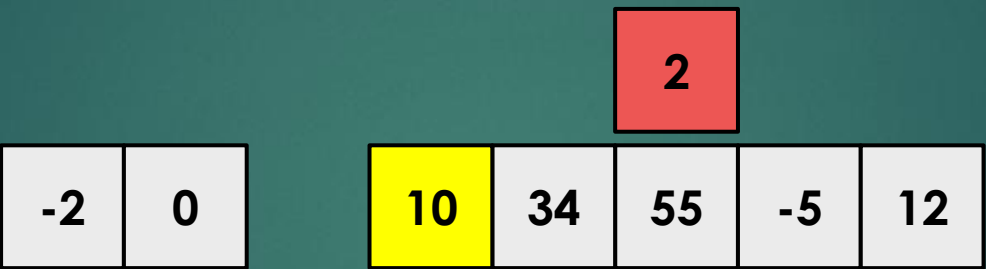
-2	0	10
----	---	----

34	55	-5	12
----	----	----	----

2

-2	0	10
----	---	----

2			
34	55	-5	12



-2	0
----	---

10	34	55	-5	12
----	----	----	----	----

2

-2	0
----	---

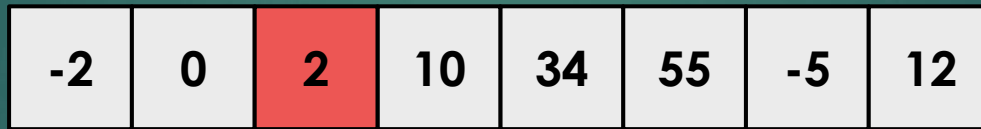
10	34	55	-5	12
----	----	----	----	----

2

-2	0
----	---

10	34	55	-5	12
----	----	----	----	----

2



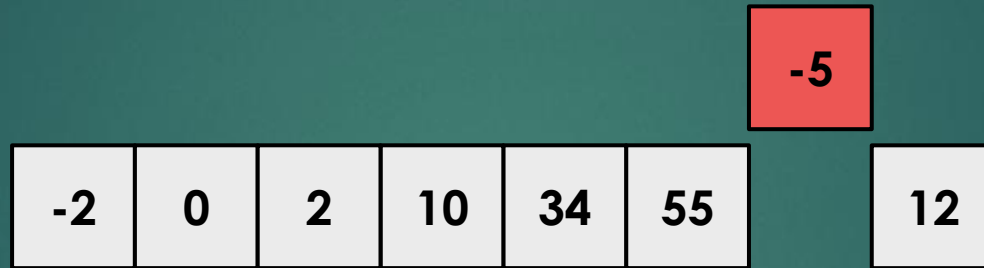
-2	0	2	10	34	55	-5	12
----	---	---	----	----	----	----	----

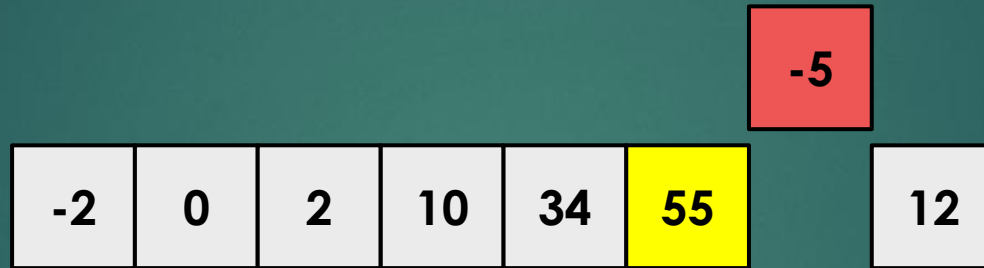


-2	0	2	10	34	55	-5	12
----	---	---	----	----	----	----	----



-2	0	2	10	34	55	-5	12
----	---	---	----	----	----	----	----





-2	0	2	10	34
----	---	---	----	----

-5	
55	12

-2	0	2	10	34
----	---	---	----	----

-5	
55	12

-2	0	2	10
----	---	---	----

	-5	
34	55	12

-2	0	2	10
----	---	---	----

	-5	
34	55	12

-2	0	2
----	---	---

10	34	55	12
----	----	----	----

-5

-2	0	2
----	---	---

10	34	55	12
----	----	----	----

-5



-2	0
----	---

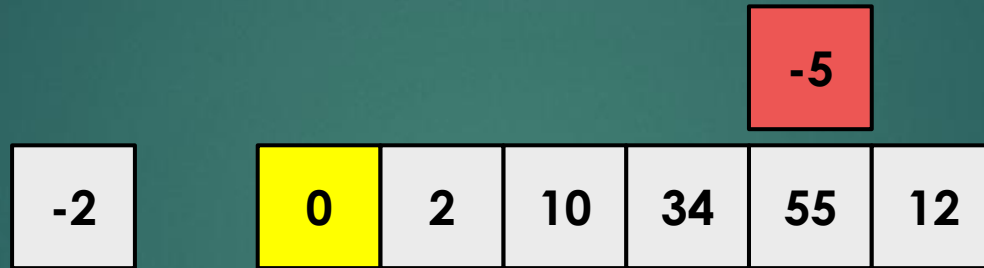
2	10	34	55	12
---	----	----	----	----

-5

-2	0
----	---

2	10	34	55	12
---	----	----	----	----

-5



-2

0	2	10	34	55	12
---	---	----	----	----	----

-5



-2	0	2	10	34	-5	12
----	---	---	----	----	----	----



-2	0	2	10	34	-5	12
----	---	---	----	----	----	----



-5	-2	0	2	10	34	55	12
----	----	---	---	----	----	----	----



-5	-2	0	2	10	34	55	12
----	----	---	---	----	----	----	----



-5	-2	0	2	10	34	55	12
----	----	---	---	----	----	----	----

-5	-2	0	2	10	34	55
----	----	---	---	----	----	----

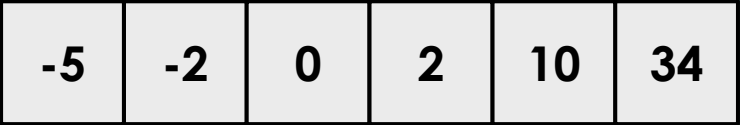
12



-5	-2	0	2	10	34	55
----	----	---	---	----	----	----



12



-5	-2	0	2	10	34
----	----	---	---	----	----



12



55

-5	-2	0	2	10	34
----	----	---	---	----	----

12
55

-5	-2	0	2	10
----	----	---	---	----

	12
34	55

-5	-2	0	2	10
----	----	---	---	----

	12
34	55

-5	-2	0	2	10
----	----	---	---	----

	12
34	55



-5	-2	0	2	10	12	34	55
----	----	---	---	----	----	----	----



-5	-2	0	2	10	12	34	55
----	----	---	---	----	----	----	----



-5	-2	0	2	10	12	34	55
----	----	---	---	----	----	----	----

Sortiranje selekcijom (Selection Sort)

Sortiranje selekcijom (Selection Sort)

- Za razliku od algoritma Bubble Sort koji postavlja najveće elemente na kraj niza, **Selection Sort** radi obrnuto, postavlja najmanje elemente na početak niza
- U svakom prolazu trenutni element niza se deklariše kao najmanji, a zatim se u ostatku niza sa desne strane traži element sa manjom vrednošću
- Ukoliko je najmanji element sa desne strane niza manji od trenutnog elementa niza, vrši se njihova zamena (swap)
- Vremenska složenost izvršavanja ovog algoritma je $O(n^2)$
- Iako algoritmi Bubble Sort i Selection Sort imaju istu vremensku složenost $O(n^2)$, Bubble Sort zahteva više zamena nego Selection Sort, jer u svakom prolazu može zameniti susedne elemente više puta dok se najveći element ne postavi na svoju ispravnu poziciju, dok Selection Sort izvršava samo jednu zamenu po prolazu, pomerajući najmanji (ili najveći) element direktno na njegovo ispravnu poziciju

Selection Sort

Selection Sort sortiranje - postupak

arr[] = {5, 2, 9, 1, 5}

Prolazak 1:

[**5**, 2, 9, 1, 5]: min=0 (5), u ostatku niza najmanji broj je 1

[**1**, 2, 9, **5**, 5]: Nakon zamene 1 i 5

Nakon prvog prolaska spoljašnje petlje algoritma, sortiran je nulti element niza (broj 1). Po istom principu dalje se sortira ostatak niza, gde se u svakoj sledećoj iteraciji spoljašnje petlje nastavlja od sledećeg elementa

Prolazak 2:

[1, **2**, 9, **5**, 5]: min=1 (2), u ostatku niza najmanji broj je 5

[**1**, 2, 9, **5**, 5]: Zamena 2 i 5 nije potrebna

Prolazak 3:

[1, 2, **9**, **5**, 5]: min=2 (9), u ostatku niza najmanji broj je 5

[1, 2, **5**, **9**, 5]: Nakon zamene 9 i 5

Prolazak 4:

[1, 2, 5, **9**, **5**]: min=3 (9), u ostatku niza najmanji broj je 5

[1, 2, **5**, **5**, 9]: Nakon zamene 9 i 5

Selection Sort – pseudokod algoritma

Algoritam	Selection sort.
-----------	-----------------

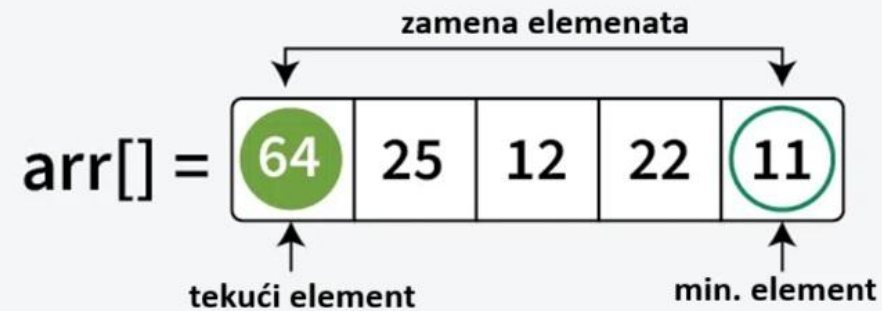
```
1: procedure SELECTION-SORT(niz)
2:   for  $i \leftarrow 0 \dots \text{niz.veličina-niza} - 1$  do
3:      $\text{min} \leftarrow i$ 
4:     for  $j \leftarrow i + 1 \dots \text{niz.veličina-niza}$  do
5:       if  $\text{niz}[j] < \text{niz}[\text{min}]$  then
6:          $\text{min} \leftarrow j$ 
7:       end if
8:     end for
9:     if  $\text{min} \neq i$  then
10:      zamijeni  $\text{niz}[\text{min}]$  i  $\text{niz}[i]$ 
11:    end if
12:  end for
13: end procedure
```

▷ najmanji element je na početku niza
▷ proveriti ostatak niza

Selection Sort

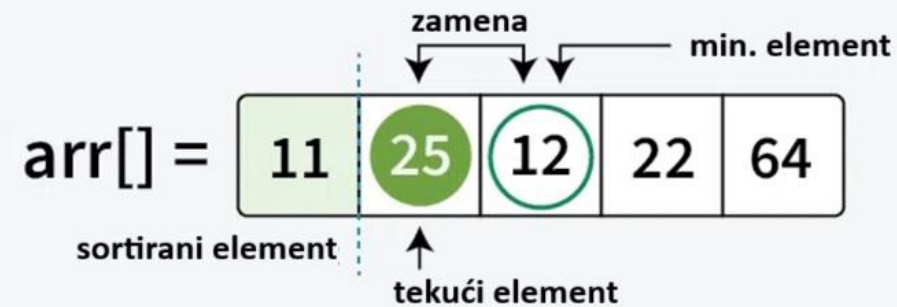
01
Step

Početak od prvog elementa na indexu 0,
pronaći najmanji element u nesortiranom ostatku
reda i zameniti 11 sa tekućim elementom 64



02
Step

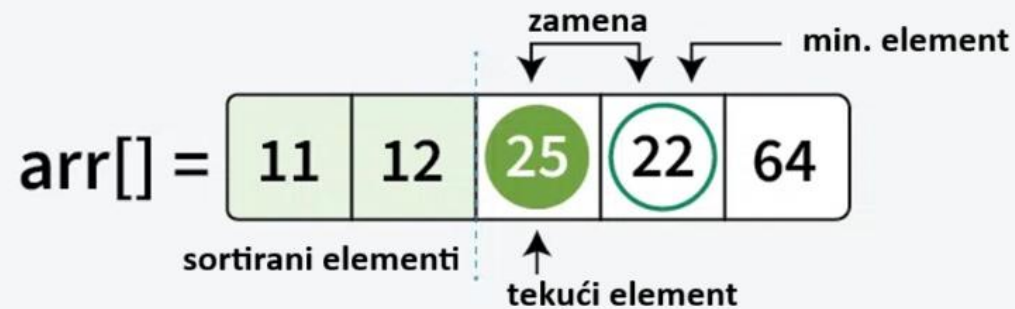
Prelazak na sledeći element sa indeksom 1 (25).
Pronaći najmanji element u nesortiranom podnizu,
i zameniti (12) sa tekućim elementom (25)



Selection Sort

03
Step

Prelazak na sledeći element sa indeksom 2 (25).
Pronaći najmanji element u nesortiranom podnizu,
zameniti (22) sa tekućim elementom (25)



04
Step

Prelazak na element sa indeksom 3 (25).
Pronaći najmanji element u nesortiranom podnizu,
zameniti (25) sa tekućim elementom (25)



Selection Sort

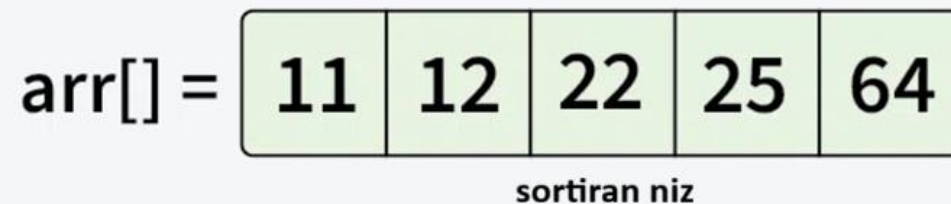
05
Step

Prelazak na element sa indeksom 4 (64).
pronaći najmanji element u nesortiranom podnizu,
zameniti (64) sa tekućim elementom (64)



06
Step

U poslednjem koraku niz je sortiran



Selection Sort

```
1  # Selection sort algoritam....
2
3  def selection_sort(array):
4      tekuci_idx = 0
5      while tekuci_idx < len(array) - 1:
6          najmanji_idx = tekuci_idx
7          for i in range(tekuci_idx + 1, len(array)):
8              if array[i] < array[najmanji_idx]:
9                  najmanji_idx = i
10
11         array[tekuci_idx], array[najmanji_idx] = ~
12         array[najmanji_idx], array[tekuci_idx]
13         tekuci_idx += 1
14
15     return array
16
17 arr = [9, 7, 5, 3, 1]
18 print(f"Ulaz: arr = {arr}")
19 print("Izlaz:", selection_sort(arr))
```

● Ulaz: arr = [9, 7, 5, 3, 1]
Izlaz: [1, 3, 5, 7, 9]

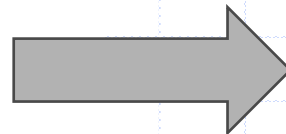
Selection Sort algoritam počinje sa spoljašnjom petljom koja predstavlja granicu između sortiranih i nesortiranih delova niza, u svakoj iteraciji pronalazi se najmanji element u nesortiranom delu i zamenjuje se sa indeksom tekućeg elementa

- Promenljiva `tekuci_idx` prati trenutni indeks koji razdvaja sortirane i nesortirane delove niza
- Promenljiva `najmanji_idx` je inicijalizovana na `tekuci_idx`, što predstavlja indeks elementa pronađenog kao najmanji
- Unutrašnja for petlja počinje od `current_idx + 1` i iterira se kroz preostali nesortirani deo niza. Ona upoređuje svaki element sa elementom na `najmanji_idx` i ažurira `najmanji_idx` ako se pronađe manji element
- Nakon nalaženja najmanjeg elementa, vrši se zamena elementa na `tekuci_idx` sa elementom na indeksu `najmanji_idx`
- `tekuci_idx` je inkrementiran i process se nastavlja dok se ne sortira ceo niz

Selection Sort

```
1  # Selection sort algoritam....
2
3  def selectionSort(arr):
4      n = len(arr)
5      for i in range(n - 1):
6
7          # Tekući element je najmanji...
8          min_idx = i
9
10         # Iteracijakroz nesortirani deo niza
11         # da se pronađe stvarni najmanji element
12         for j in range(i + 1, n):
13             if arr[j] < arr[min_idx]:
14
15                 # Ažuriranje min_idx ako je manji element
16                 # pronađen
17                 min_idx = j
18
19         # Premestiti najmanji element na njegovu
20         # ispravnu poziciju
21         arr[i], arr[min_idx] = arr[min_idx], arr[i]
22
23 if __name__ == "__main__":
24     arr = [64, 25, 12, 22, 11]
25
26     selectionSort(arr)
27
28     for val in arr:
29         print(val, end=" ")
```

- Selection Sort algoritam ima vremensku složenost $O(n^2)$ jer sadrži dve **for** petlje:
 - a) Prva **for** petlja je izbor elemenata niza redom od početka sa složenošću $O(n)$
 - b) Druga **for** petlja je poređenje elementa niza sa svakim drugim elementom, sa složenošću $O(n)$
- Sveukupna vremenska složenost Selection Sort algoritma je $O(n) * O(n) = O(n^2)$
- Selection Sort algoritam ima manji broj zamena (memorijskih upisa) u poređenju sa drugim algoritmima
- Nije stabilan algoritam jer ne zadržava relativni poredak istih elemenata niza

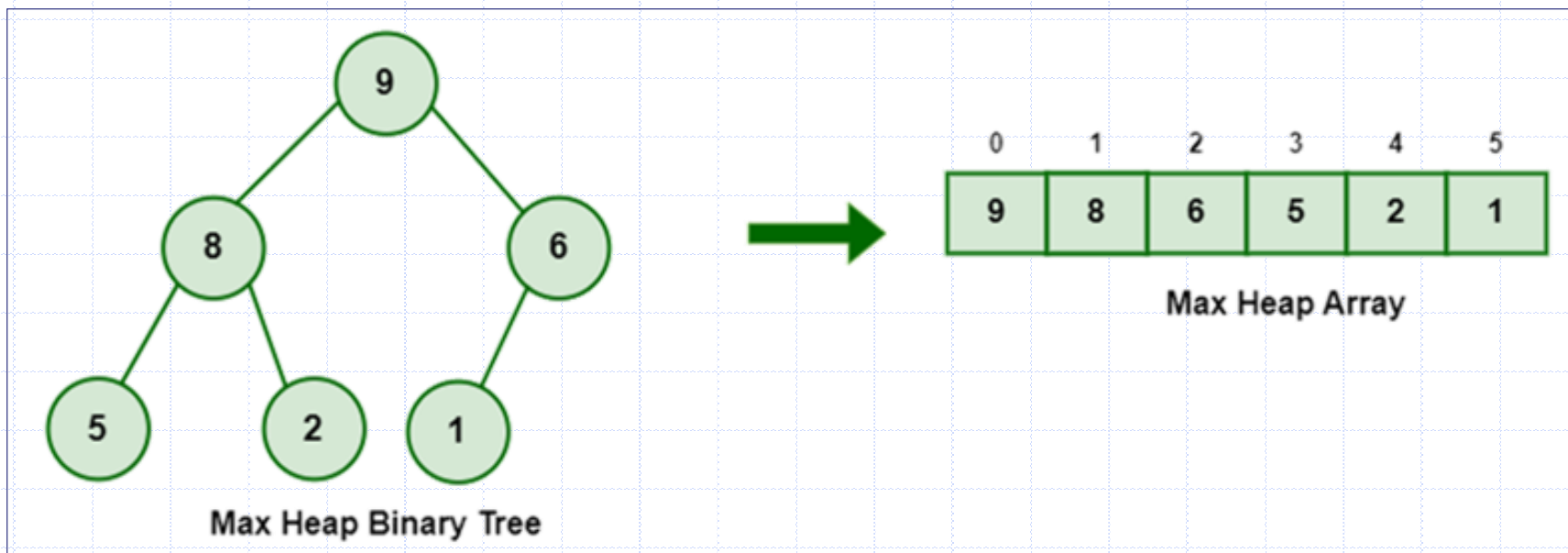


11 12 22 25 64

Sortiranje hipa (Heap Sort)

Sortiranje binarnog hipa (Heap Sort)

- **Heap Sort** algoritam sortiranja zasniva se na strukturi podataka **binarni hip** (binary heap)
- Binarni hip je kompletno binarno stablo sa elementima u parcijalno uređenom redosledu, tako da je svaki ključ nekog elementa koji je roditelj, manji od ključa elementa levog, odnosno desnog potomka
- Kod **max hipa**, korenski čvor stabla je element sa najvećim ključem, a roditelj je uvek veći ili jednak od svih ključeva elemenata potomaka

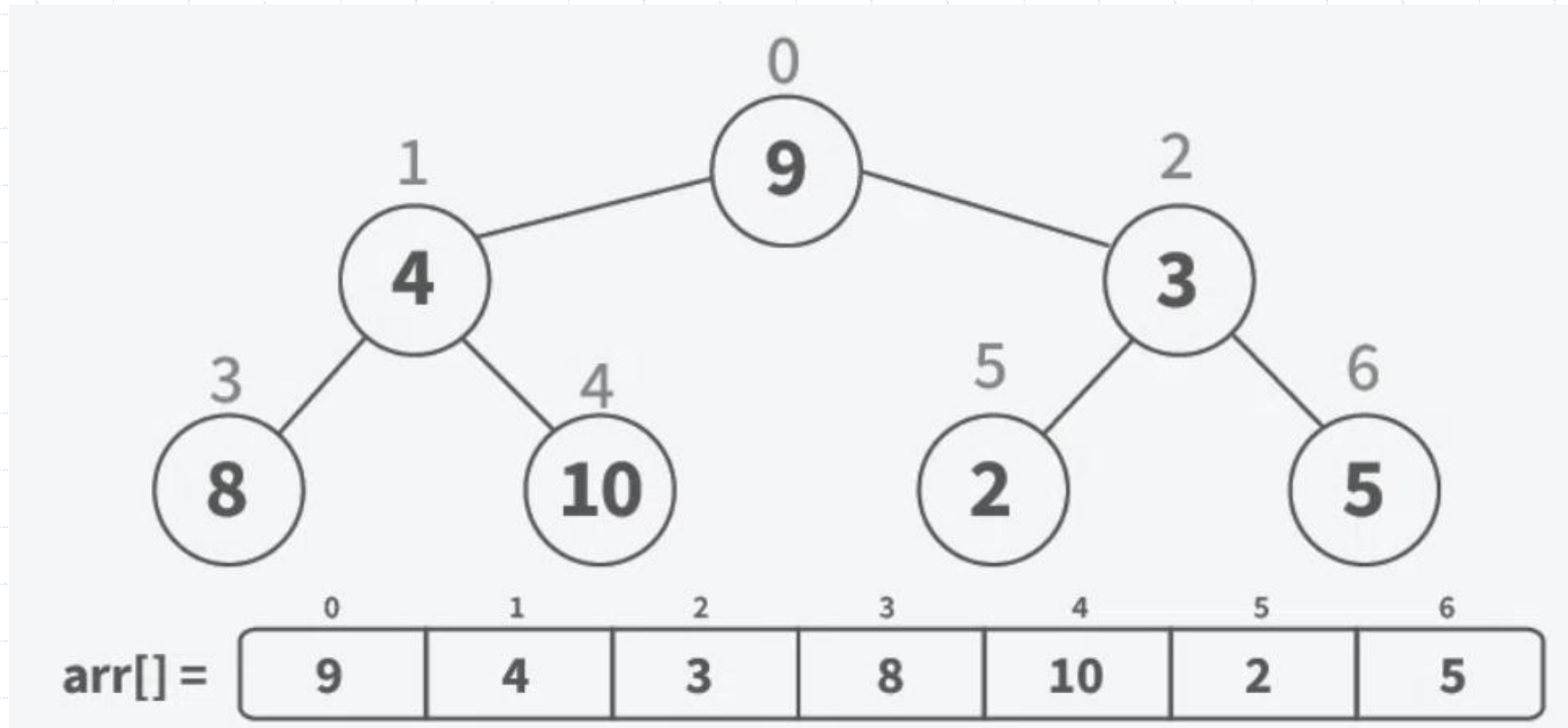


Heap Sort

- Heap Sort algoritam sortiranja zasnovan je na poređenju elemenata u strukturi binarnog hipa
- Predstavlja optimizovanu verziju Selection Sort algoritma sortiranja
- Algoritam više puta pronalazi maksimalni (ili minimalni) element i zamenjuje ga sa poslednjim (ili prvim) elementom
- Proces se ponavlja za preostale elemente dok se niz ne sortira
- Vremenska složenost Heap Sort algoritma je $O(n \log_2 n)$ jer je za formiranje binarnog hipa potrebno $O(n)$ vremena, a za svaki od $(n-1)$ poziva binarnog hipa treba $O(\log_2 n)$ vremena
- Heap Sort algoritam: Prvo se niz konvertuje u max-hip koristeći heapify princip (elementi niza se preuređuju tako da prate princip formiranja hipa), a zatim se jedan po jedan uklanjaju korenski čvorovi max-hipa i zamenjuju sa poslednjim elementom nesortiranog niza, postupak se ponavlja dok se ne dođe do kraja forme max-hipa

Heap Sort

- KORAK 1: Potrebno je prikazati niz kao kompletno binarno stablo, za niz veličine n koren je na indeksu 0 , levi potomak elementa na indeksu i je na $2i+1$, a desni potomak je na $2i+2$

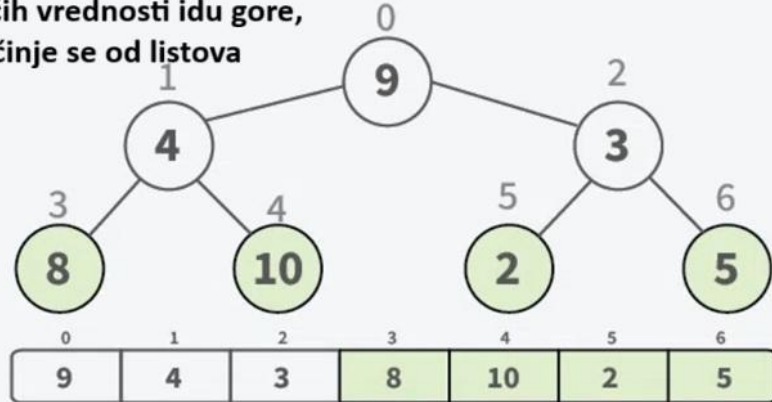


Heap Sort

- KORAK 2:** Formiranje **max-hip** stabla (**heapfy binarno stablo**) nad ulaznim nizom $arr=[9, 4, 3, 8, 10, 2, 5]$

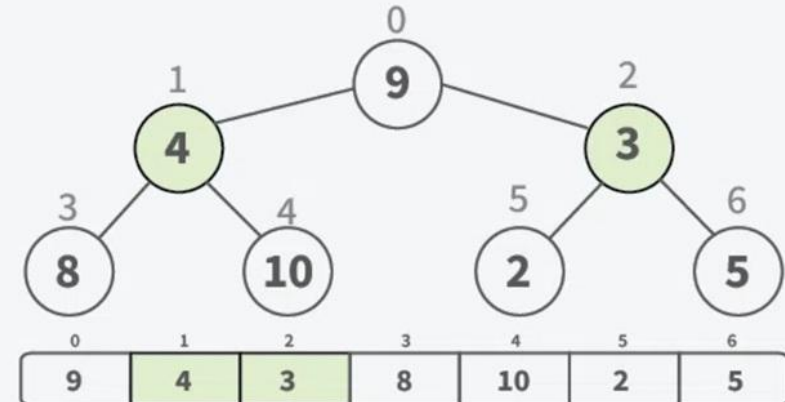
01
Step

Porediti svaki čvor sa potomcima, roditeljski čvorovi treba da budu veći, čvorovi manje vrednosti odlaze dole, dok čvorovi većih vrednosti idu gore, počinje se od listova



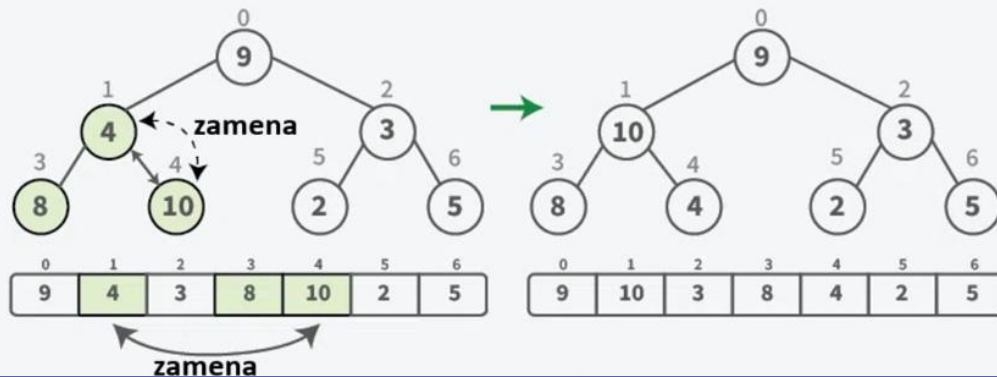
02
Step

Polazimo od sloja 1 gde su čvorovi 3 i 4



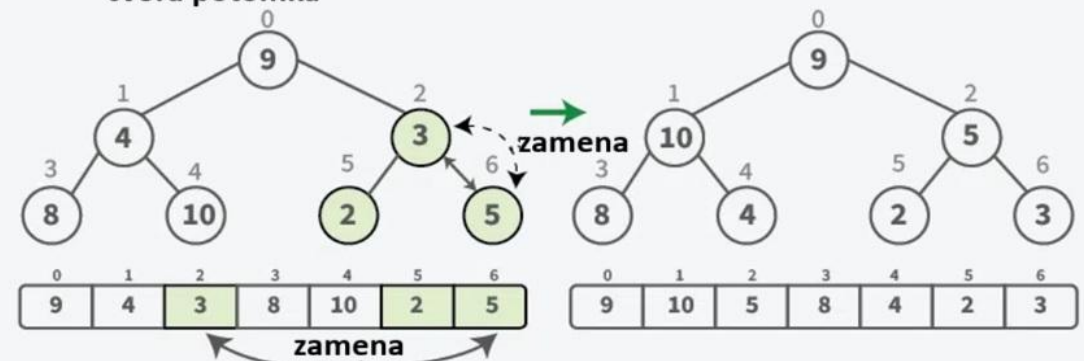
03
Step

Roditeljski čvor 4 je manji od čvora potomka (10), pa oni menjaju mesta jer čvor roditelja treba da bude veći od potomka



04
Step

Roditeljski čvor 3 je manji od čvora potomka 5, oni menjaju mesta jer čvor roditelja treba da bude veći od čvora potomka

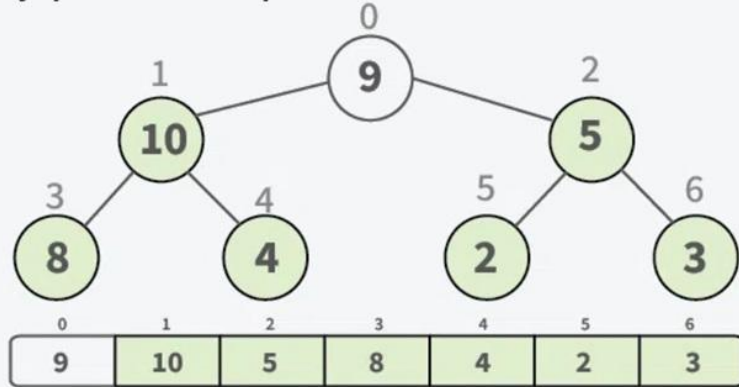


Heap Sort

- KORAK 2:** Formiranje **max-hip** stabla (**heapfy binarno stablo**) nad ulaznim nizom $arr=[9, 4, 3, 8, 10, 2, 5]$

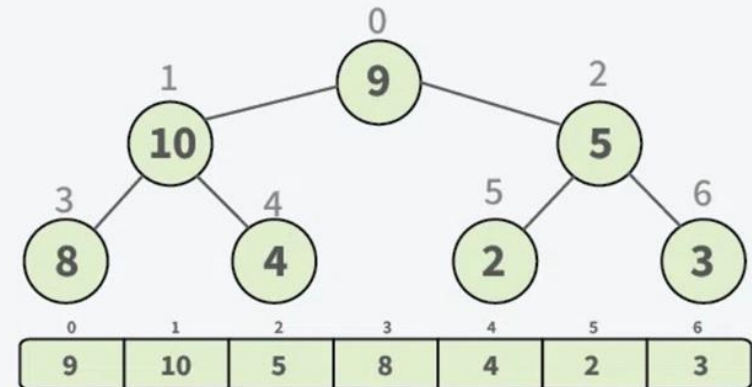
05
Step

Nakon završenog nivoa 1 binarnog stabla, imamo dva manja pravilna max-hipa



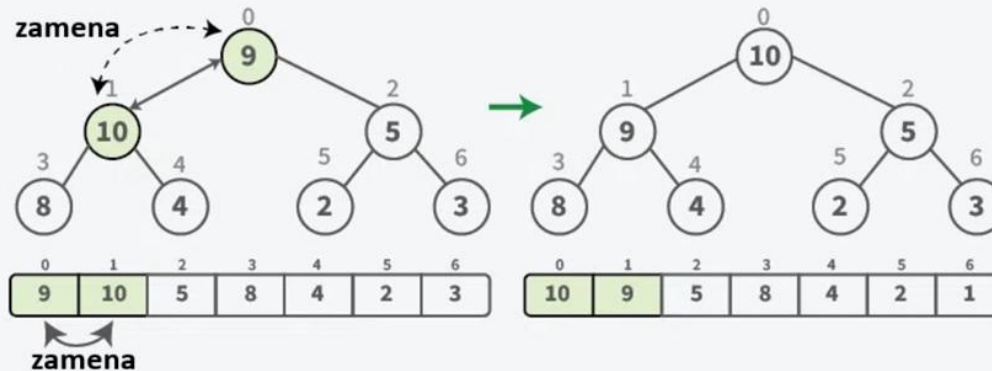
06
Step

Prelazak na sloj 0 gde je čvor korena 9



07
Step

Čvor korena 9 je manji od njegovog čvora potomka 10, tako da oni menjaju mesta kako bi roditeljski čvor bio veći od potomka

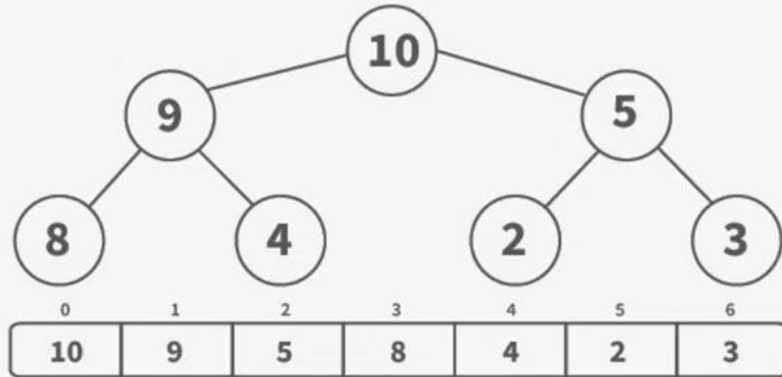


Heap Sort

- KORAK 3:** Sortiranje niza tako što se najveći element postavi na kraj nesortiranog niza

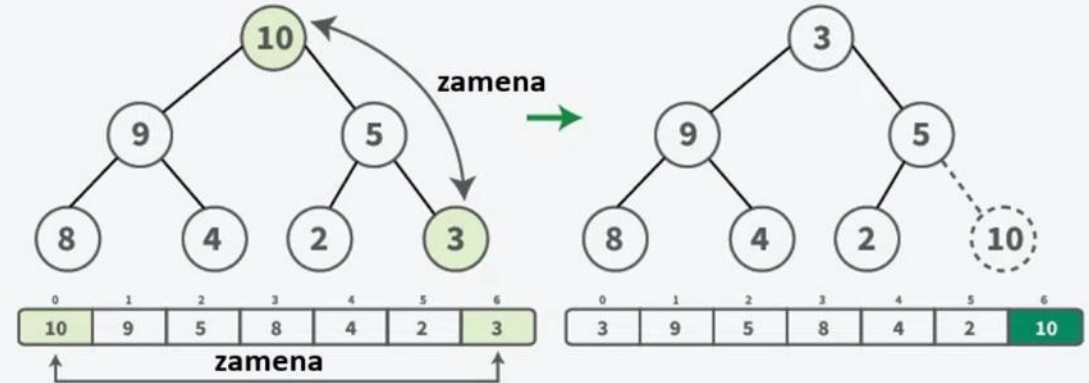
01
Step

Početna forma nesortiranog niza prikazanog kao max-heap



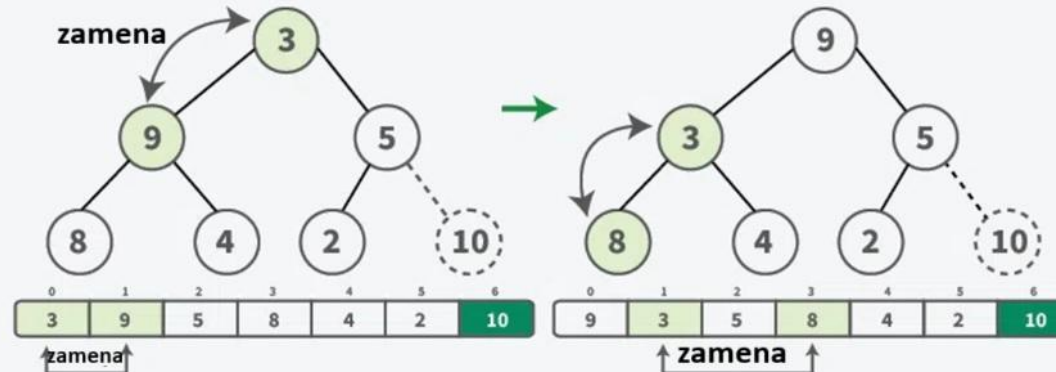
02
Step

Zameniti max. element (10) sa poslednjim elementom nesortiranog niza (3), smanjiti veličinu hipa za 1, poslednji element je sortiran



03
Step

Čvor korena narušava max-heap princip, i on se zamenjuje sa najvećim čvorom potomkom (9), a pošto čvor korena 3 ima još jednog većeg potomka (8), zamenjuje se sa njim

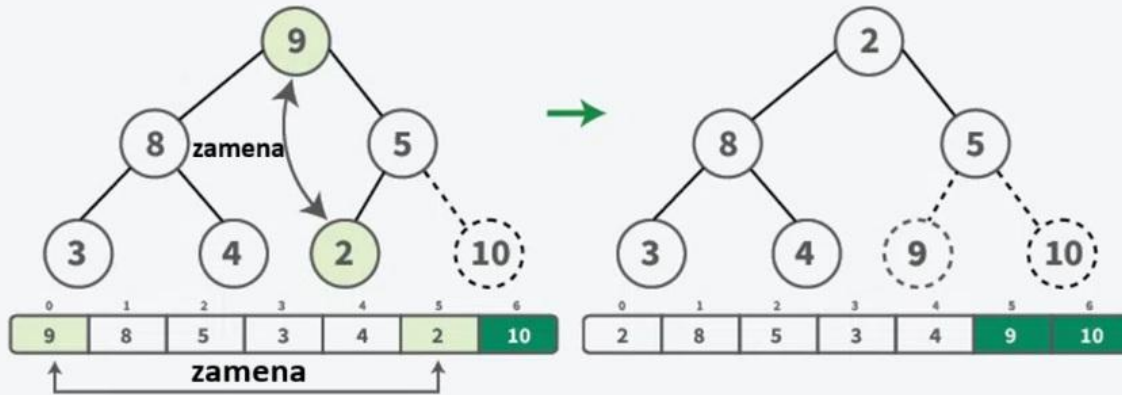


Heap Sort

- KORAK 3:** Sortiranje niza tako što se najveći element postavi na kraj nesortiranog niza

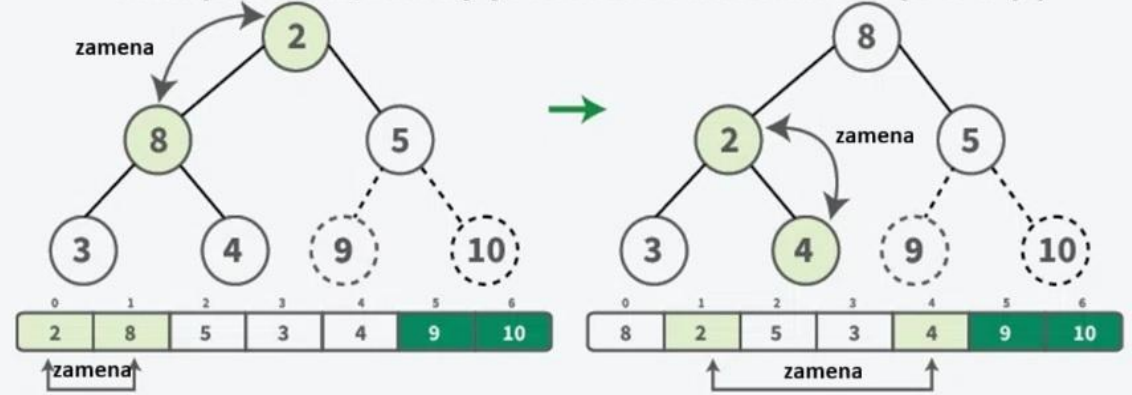
04
Step

Formiran je max-heap. Zameniti max. element (9) sa poslednjim elementom nesortiranog niza (2), smanjiti veličinu hipa za 1, element 9 je sortiran i dalje se ne razmatra



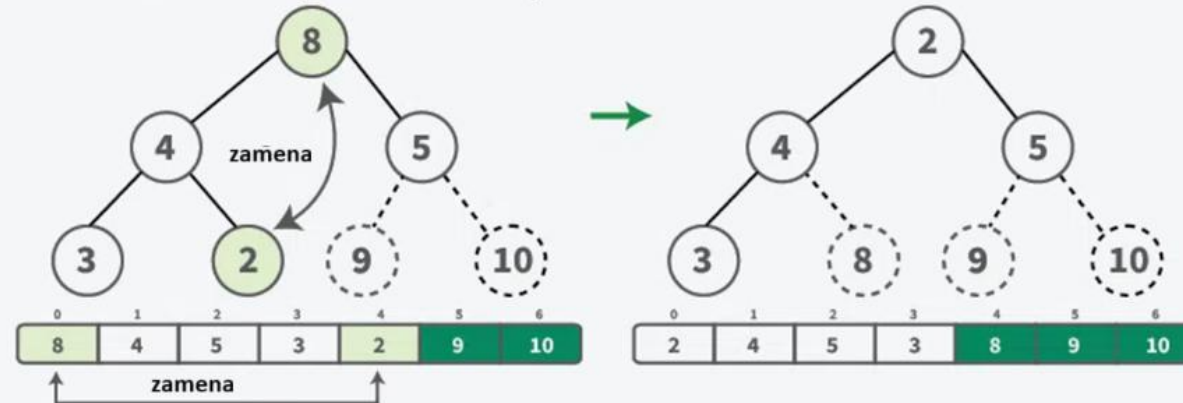
05
Step

Čvor korena opet narušava max-heap princip, on se zamenjuje sa najvećim čvorom potomkom (8), a pošto čvor korena 2 ima još većih potomaka, zamenjuje se sa sledećim većim a to je čvor (4)



06
Step

Formiran je max-heap. Zameniti max. element (8) sa poslednjim elementom nesortiranog niza (2), smanjiti veličinu hipa za 1, element 8 je sortiran i dalje se ne razmatra



Heap Sort - kod

```
1 # Heap sort algoritam....
2 def heapify(arr, n, i):
3     largest = i
4     left = 2 * i + 1
5     right = 2 * i + 2
6
7     if left < n and arr[left] > arr[largest]:
8         largest = left
9
10    if right < n and arr[right] > arr[largest]:
11        largest = right
12
13    if largest != i:
14        arr[i], arr[largest] = arr[largest], arr[i]
15        heapify(arr, n, largest)
16
17 def heap_sort(array):
18     n = len(array)
19
20     # Formiranje max heapa
21     for i in range(n // 2 - 1, -1, -1):
22         heapify(array, n, i)
23
24     # Izdvajanje elemenata iz hipa one by one
25     for i in range(n - 1, 0, -1):
26         array[i], array[0] = array[0], array[i]
27         heapify(array, i, 0)
28     return array
29
30 arr = [1, 2, 3, 4, 5]
31 print(f"Input: arr = {arr}")
32 print("Output:", heap_sort(arr))
```

Heap Sort algoritam sortiranja zasnovan je na poređenju elemenata u strukturi podataka binarni hip. Prvo se formira max-hip od ulaznog niza, a zatim više puta izdvaja najveći element iz hipa, koji se zamenjuje sa poslednjim elementom hipa, tako da se smanjuje veličina hipa. Proces se nastavlja sve dok se ceo niz ne sortira.

- Funkcija `heapify` kreira max-hip iz nesortiranog niza. Ona uzima tri argumenta: niz, veličinu hipa (n) i indeks tekućeg elementa (i). Funkcija poredi i -ti element sa njegovim levim i desnim potomkom (ako postoje) i zamenjuje ga sa većim potomkom. Proces se rekurzivno ponavlja kako bi se osiguralo da se svojstvo hipa održi

- Funkcija `heap_sort` prvo formira max-hip iz ulaznog niza, pozivom funkcije `heapify` na svim čvorovima koji nisu listovi, počevši od poslednjeg čvora koji nije list, pa do korena. Nakon formiranja max-hipa, funkcija više puta izdvaja najveći element iz hipa, zamenjujući ga sa poslednjim elementom i smanjujući veličinu hipa. Ovaj korak se izvršava obrnutim redosledom, da bi se niz sortirao u rastućem poretku

- Funkcija `heap_sort` vraća sortirani niz

Heap Sort - kod

```
1 # Heap sort algoritam....
2
3 # To heapify a subtree rooted with node i
4 def heapify(arr, n, i):
5
6     # Inicijalizacija najvećeg kao root
7     largest = i
8
9     # left index = 2*i + 1
10    l = 2 * i + 1
11
12    # right index = 2*i + 2
13    r = 2 * i + 2
14
15    # Ako je left child veći od roota
16    if l < n and arr[l] > arr[largest]:
17        largest = l
18
19    # Ako je right child veći od najvećeg ...
20    if r < n and arr[r] > arr[largest]:
21        largest = r
22
23    # Ako najveći nije root
24    if largest != i:
25        arr[i], arr[largest] = arr[largest], arr[i]
26
27    # Rekurzivni heapify za podstablo
28    heapify(arr, n, largest)
```

```
29
30 # Glavna funkcija za izvršavanje heap sorta
31 def heapSort(arr):
32     n = len(arr)
33
34     # Formiranje heapa (preuređenje vektora)
35     for i in range(n // 2 - 1, -1, -1):
36         heapify(arr, n, i)
37
38     # Jedan po jedan izdvojiti elemente iz heapa
39     for i in range(n - 1, 0, -1):
40
41         # Prebaciti tekući root na kraj
42         arr[0], arr[i] = arr[i], arr[0]
43
44         # Poziv max heapify za redukovani heap
45         heapify(arr, i, 0)
46
47 if __name__ == "__main__":
48     arr = [9, 4, 3, 8, 10, 2, 5]
49
50     heapSort(arr)
51
52     for i in range(len(arr)):
53         print(arr[i], end=" ")
```

